开发者社区

DTK

DTK 26.10
rocBLAS 库使用手册

DTK-261000-025-0915-940

目　录

1 rocBLAS 库简介 1
2 环境要求 1
3 快速开始 1
3.1 直接使用 DTK 集成的 rocBLAS（推荐） 1
3.2 从源码编译 rocBLAS 1
4 编程模型与核心概念 2
4.1 调用流程概述 2
4.2 Handle 的作用与管理 3
4.3 内存管理 3
4.4 数据布局 4
4.5 Batch 模式简介 4
5 常用 API 详解 4
5.1 rocBLAS 数据类型 4
5.1.1 rocblas_int 4
5.1.2 rocblas_stride 4
5.1.3 rocblas_half/rocblas_bfloat16 5
5.1.4 rocblas_float_complex/rocblas_double_complex 5
5.2 rocBLAS 枚举 5
5.2.1 计算控制类 5
5.2.2 类型与精度控制 6
5.2.3 日志与调试控制 6
5.2.4 调试与容错控制 6
5.3 rocBLAS 辅助函数 7
5.3.1 基本操作 7
5.3.2 数据处理 8
5.3.3 设备内存管理 8
5.3.4 版本查询 8
5.4 rocBLAS 一级函数 9

开发者社区

5.4.1 amax 函数族 9
5.4.2 amin 函数族 10
5.4.3 asum 函数族 11
5.4.4 axpy 函数族 13
5.4.5 copy 函数族 15
5.4.6 dot 函数族 16
5.4.7 nrm2 函数族 19
5.4.8 rot 函数族 21
5.4.9 rotg 函数族 23
5.4.10 rotm 函数族 25
5.4.11 rotmg 函数族 26
5.4.12 scal 函数族 27
5.4.13 swap 函数族 29
5.5 rocBLAS 二级函数 30
5.5.1 gbmv 函数族 30
5.5.2 gemv 函数族 33
5.5.3 ger 函数族 36
5.5.4 sbmv 函数族 39
5.5.5 spmv 函数族 40
5.5.6 spr 函数族 42
5.5.7 spr2 函数族 43
5.5.8 symv 函数族 44
5.5.9 syr 函数族 47
5.5.10 syr2 函数族 49
5.5.11 tbmv 函数族 51
5.5.12 tbsv 函数族 53
5.5.13 tpmv 函数族 55
5.5.14 tpsv 函数族 57
5.5.15 trmv 函数族 59

5.5.16 trsv 函数族.....61
5.5.17 hemv 函数族.....63
5.5.18 hbmv 函数族.....65
5.5.19 hpmv 函数族.....66
5.5.20 her 函数族.....68
5.5.21 her2 函数族.....69
5.5.22 hpr 函数族.....70
5.5.23 hpr2 函数族.....71
5.6 rocBLAS 三级函数.....73
5.6.1 gemm 函数族.....73
5.6.2 symm 函数族.....76
5.6.3 syrk 函数族.....79
5.6.4 syr2k 函数族.....81
5.6.5 syrkx 函数族.....84
5.6.6 trmm 函数族.....87
5.6.7 trsm 函数族.....89
5.6.8 hemm 函数族.....92
5.6.9 herk 函数族.....94
5.6.10 her2k 函数族.....95
5.6.11 herkx 函数族.....97
5.6.12 trtri 函数族.....99
5.7 rocBLAS 扩展函数.....100
5.7.1 Axpy.....100
5.7.2 Dot.....101
5.7.3 Dotc.....102
5.7.4 Nrm2.....103
5.7.5 Rot.....104
5.7.6 Scal.....105
5.7.7 Gemm.....106
5.7.8 Trsm.....107

5.7.9 Geam 108
5.7.10 Sdgmm 110
5.7.11 Syrk_Ex 112
6 rocBLAS 基准测试 112
6.1 rocblas-bench 112
6.2 rocblas-gemm-tune 118
6.3 rocblas-test 118
7 Demo 案例 119
7.1 示例代码 119
7.2 编译指令 121
7.3 输出结果 121

开发者社区

版本更新：

- DTK-25.04.2：
  - 创建文档。

开发者社区

1 rocBLAS 库简介

rocBLAS 是一套面向 GPU 平台的高性能基础线性代数子程序（BLAS）库，实现了 BLAS Level 1、Level 2 和 Level 3 中常见的向量与矩阵操作，如向量加法、矩阵-向量乘法、矩阵-矩阵乘法等。该库使用 HIP 编程模型实现，具备良好的可扩展性与高效的内核调度机制，适用于通用加速计算场景。其支持多种精度格式，包括单精度、双精度、半精度（FP16/BF16）以及整数类型，适配科学计算、训练推理等多样化应用需求。其内部通过融合计算、内核缓存与调度优化等手段，提升整体执行效率。库接口设计兼容传统 BLAS 标准，同时支持批量计算与异步执行，方便集成于高层框架或自定义计算流程中。rocBLAS 致力于在保持功能完备性的同时，实现更优的性能与稳定性。

2 环境要求

硬件要求：DCU 以及配套驱动；

软件栈要求：DTK（优先使用高版本 DTK）；

3 快速开始

3.1 直接使用 DTK 集成的 rocBLAS（推荐）

在大多数情况下，用户无需自行编译 rocBLAS 源码，DTK（Device ToolKit）或 ROCm 平台默认已集成 rocBLAS 库。只需在构建工程时正确链接对应库和头文件即可。

依赖项：已加载的 DTK。

3.2 从源码编译 rocBLAS

若需修改源码或集成至自定义平台，可选择从源码手动编译 rocBLAS。

依赖项：

- CMake 3.23+；
- Python3；
- DTK 工具链（hipcc 等）；
- Tensile；

示例步骤：

```
cd rocBLAS
# 配置构建目录
mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=./install \
        -DCMAKE_BUILD_TYPE=Release \
        -DBUILD_WITH_TENSILE=ON \
-DTensile_TEST_LOCAL_PATH=./tensile-dtk-25.04.1 \
        -DTENSILE_LIBRARY_FORMAT=msgpack \
        -DBUILD_CLIENTS_TESTS=OFF \
        -DBUILD_CLIENTS_SAMPLES=OFF

# 构建并安装
make -j$(nproc)
make install
```

4 编程模型与核心概念

本章介绍 rocBLAS 的基本调用流程、核心对象 rocblas_handle、数据管理方式，以及同步行为等核心概念。掌握这些内容，有助于理解 API 的使用语义和潜在性能特性。

4.1 调用流程概述

一个典型的 rocBLAS 调用流程如下：

1) 初始化 rocBLAS 库
2) 创建 rocblas_handle
3) 分配并准备 GPU 端数据（A/B/C 等）
4) 设置参数并调用 rocBLAS 接口（如 GEMM、AXPY）
5) 等待计算完成（可选同步）
6) 拷贝结果回 Host（如有需要）
7) 释放资源并销毁 handle

简单示例代码如下：

开发者社区

```
rocblas_handle handle;
rocblas_create_handle(&handle);
rocblas_sgemm(handle,
                transA, transB,
                m, n, k,
                &alpha,
                d_A, lda,
                d_B, ldb,
                &beta,
                d_C, ldc);
rocblas_destroy_handle(handle);
```

4.2 Handle 的作用与管理

rocblas_handle 是 rocBLAS 中用于管理上下文状态的核心对象，类似于 cuBLAS 的 handle，内部记录了：

1) 当前使用的 HIP stream
2) 内部内存缓存指针
3) 库上下文信息

- 创建与销毁

```
rocblas_handle handle;
rocblas_create_handle(&handle);
// ...
rocblas_destroy_handle(handle);
```

- 自定义 HIP Stream

```
hipStream_t userStream;
hipStreamCreate(&userStream);
rocblas_set_stream(handle, userStream);
```

4.3 内存管理

rocBLAS 所有计算均在 GPU 设备端执行，因此：

1) 所有矩阵指针（如 A/B/C）必须是 GPU device 指针

2) 参数 alpha、beta 也必须是 GPU 指针，不能传 host 上的常量地址

3) 建议使用 hipMalloc() / hipMemcpy() 管理内存

4.4 数据布局

rocBLAS 默认采用 列主序（Column-major） 的数据布局，列主序的含义：

- 内存中按 列优先 存储：矩阵中相邻的列元素在内存中是连续的。
- 例如矩阵 A=

$$\begin{bmatrix} a_{00}, a_{01} \\ a_{10}, a_{11} \end{bmatrix}$$

在列主序下的内存布局为：[a00, a10, a01, a11]

4.5 Batch 模式简介

对于多个矩阵批量计算场景（如 Transformer 推理、Batched GEMM），rocBLAS 支持：

- Batched 模式：传入矩阵指针数组
- Strided Batched 模式：传入起始地址 + 步长，适合内存连续的情况

5 常用 API 详解

5.1 rocBLAS 数据类型

5.1.1 rocblas_int

```
typedef int32_t rocblas_int;
```

· 代表维度、偏移等整数索引。

· 理论支持 LP64（32-bit int）或 ILP64（64-bit int）架构，但 ILP64 模式未经过测试，建议默认使用 int32_t。

5.1.2 rocblas_stride

typedef int64_t rocblas_stride;

· 用于 strided batched 函数中，指定每个矩阵或向量之间在内存中的偏移（字节数或元素数）。

· 支持大范围索引，使用 int64_t 以覆盖大批量输入。

5.1.3 rocblas_half/rocblas_bfloat16

rocblas_half：半精度浮点（fp16），一般对应_half_类型；

rocblas_bfloat16：Brain 浮点格式；

float： 单精度浮点；

double：双精度浮点。

5.1.4 rocblas_float_complex/rocblas_double_complex

```
typedef struct {
    float real;
    float imag;
} rocblas_float_complex;

typedef struct {
    double real;
    double imag;
} rocblas_double_complex;
```

· rocblas_float_complex 表示具有单精度实部和虚部的复数。

· rocblas_double_complex 表示具有双精度实部和虚部的复数。

· 用于复数矩阵操作的 GEMM、TRSM 等函数族。

5.2 rocBLAS 枚举

5.2.1 计算控制类

1) rocblas_operation 控制矩阵是否转置
   - rocblas_operation_none：不转置
   - rocblas_operation_transpose：转置
   - rocblas_operation_conjugate_transpose：共轭转置
2) rocblas_side 指定在操作中矩阵 A 所处的位置
   - rocblas_side_left：A 在左侧
   - rocblas_side_right：A 在右侧

- rocblas_side_both：双侧作用（如对称性）

3) rocblas_fill 指定三角矩阵引用的是哪个部分
   - rocblas_fill_upper：上三角
   - rocblas_fill_lower：下三角
   - rocblas_fill_full：完整矩阵
4) rocblas_diagonal 指示三角矩阵是否为单位对角
   - rocblas_diagonal_unit：单位矩阵（对角为 1）
   - rocblas_diagonal_non_unit：非单位矩阵（对角需要计算）

5.2.2 类型与精度控制

rocblas_datatype 用于指定输入/输出数据的精度或格式：

实数类型：rocblas_datatype_f16_r, f32_r, f64_r, bf16_r, f8_r, bf8_r；

复数类型：rocblas_datatype_f16_c, f32_c, f64_c, bf16_c；

整数类型：rocblas_datatype_i8_r, u8_r, i32_r, u32_r，i8_c, u8_c, i32_c, u32_c；

rocblas_datatype_invalid：无效类型，不能使用。

5.2.3 日志与调试控制

rocblas_layer_mode 设置日志记录模式（可组合）：

- rocblas_layer_mode_none：不记录
- rocblas_layer_mode_log_trace：函数调用追踪日志
- rocblas_layer_mode_log_bench：benchmark 复现日志
- rocblas_layer_mode_log_profile：性能配置日志

5.2.4 调试与容错控制

rocblas_status 函数执行结果状态码：

- rocblas_status_success：成功
- rocblas_status_invalid_handle：句柄无效
- rocblas_status_not_implemented：未实现
- rocblas_status_invalid_pointer：无效指针
- rocblas_status_invalid_size：无效尺寸
- rocblas_status_memory_error：内存出错

- rocblas_status_internal_error：内部故障
- rocblas_status_perf_degraded：性能下降
- rocblas_status_check_numerics_fail：数值错误（NaN / Inf）

等，共十余种状态码，可用于调试或容错控制。

5.3 rocBLAS 辅助函数

5.3.1 基本操作

1) 创建和销毁 Handle

rocblas_status rocblas_create_handle(rocblas_handle* handle)；

rocblas_status rocblas_destroy_handle(rocblas_handle handle)；

用于创建/销毁 rocBLAS 基础调用的基本运行环境。

2) 设置/获取 HIP Stream

rocblas_status rocblas_set_stream(rocblas_handle handle, hipStream_t stream)；

rocblas_status rocblas_get_stream(rocblas_handle handle, hipStream_t* stream)；

用于切换 rocBLAS 所使用的运行 stream，便于实现多流/多设备场景。

3) 指针模式

rocblas_status rocblas_set_pointer_mode(rocblas_handle handle, rocblas_pointer_mode pointer_mode)；

rocblas_status rocblas_get_pointer_mode(rocblas_handle handle, rocblas_pointer_mode* pointer_mode)；

rocblas_pointer_mode rocblas_pointer_to_mode(void* ptr)；

用于设置/查询函数调用时常量参数是位于主机还是设备端。

4) 原子操作模式

rocblas_status rocblas_set_atomics_mode(rocblas_handle handle, rocblas_atomics_mode mode)；

rocblas_status rocblas_get_atomics_mode(rocblas_handle handle, rocblas_atomics_mode* mode)；

查询/设置是否允许内部 kernel 使用 atomic 操作，这可能影响精确处理和可重环性。

5.3.2 数据处理

1) 向量操作

rocblas_status rocblas_set_vector(...)；

rocblas_status rocblas_get_vector(...)；

rocblas_status rocblas_set_vector_async(...)；

用于从主机向设备或透过 HIP stream 实现异步数据处理。

2) 矩阵操作

rocblas_status rocblas_set_matrix(...)；

rocblas_status rocblas_get_matrix(...)；

rocblas_status rocblas_set_matrix_async(...)；

rocblas_status rocblas_get_matrix_async(...)；

和 vector 操作一致，用于处理矩阵数据往处。

5.3.3 设备内存管理

1) 设备内存空间查询

rocblas_status rocblas_start_device_memory_size_query(rocblas_handle handle)；

rocblas_status rocblas_stop_device_memory_size_query(rocblas_handle handle, size_t* size)；

rocblas_status rocblas_get_device_memory_size(rocblas_handle handle, size_t* size)；

rocblas_status rocblas_set_device_memory_size(rocblas_handle handle, size_t size)；

用于预估内核操作所需内存。

2) 用户自定义内存空间

rocblas_status rocblas_set_workspace(rocblas_handle handle, void* addr, size_t size)；

bool rocblas_is_managing_device_memory(rocblas_handle handle)；

bool rocblas_is_user_managing_device_memory(rocblas_handle handle)；

用于指定用户手动分配的 GPU 内存给 rocBLAS 使用，以避免内部自动 malloc 并提升控制力。

5.3.4 版本查询

rocblas_status rocblas_get_version_string_size(size_t* len)；

开发者社区

rocblas_status rocblas_get_version_string(char* buf, size_t len)；

用于获取当前运行的 rocBLAS 库版本。

5.4 rocBLAS 一级函数

rocBLAS 一级函数执行标量、向量和向量-向量运算。

5.4.1 amax 函数族

amax 查找向量 x 中最大值元素的第一个索引。

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 中的元素数量；
3) x：存储向量 x 设备指针；
4) incx：指定 x 元素的增量；
5) result：存储结果；
6) batch_count：批次数；
7) stridex：x_i 与 x_(i+1)之间的步长；

rocblas_status rocblas_isamax(rocblas_handle handle, rocblas_int n, const float *x, ro
cblas_int incx, rocblas_int *result)

rocblas_status rocblas_idamax(rocblas_handle handle, rocblas_int n, const double *x,
rocblas_int incx, rocblas_int *result)

rocblas_status rocblas_icamax(rocblas_handle handle, rocblas_int n, const rocblas_flo
at_complex *x, rocblas_int incx, rocblas_int *result)

rocblas_status rocblas_izamax(rocblas_handle handle, rocblas_int n, const rocblas_do
uble_complex *x, rocblas_int incx, rocblas_int *result)

rocblas_status rocblas_isamax_batched(rocblas_handle handle, rocblas_int n, const fl
oat *const x[], rocblas_int incx, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_idamax_batched(rocblas_handle handle, rocblas_int n, const d
ouble *const x[], rocblas_int incx, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_izamax_batched(rocblas_handle handle, rocblas_int n, const ro
cblas_double_complex *const x[], rocblas_int incx, rocblas_int batch_count, rocblas_

开发者社区

int *result)

rocblas_status rocblas_isamax_strided_batched(rocblas_handle handle, rocblas_int n, const float *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_idamax_strided_batched(rocblas_handle handle, rocblas_int n, const double *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_icamax_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_izamax_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, rocblas_int *result)

5.4.2 amin 函数族

amin 找到向量 x 中最小幅值元素的第一个索引：

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 中的元素数量；
3) x：存储向量 x 设备指针；
4) incx：指定 x 元素的增量；
5) result：存储结果；
6) batch_count：批次数；
7) stridex：x_i 与 x_(i+1)之间的步长；

rocblas_status rocblas_isamin(rocblas_handle handle, rocblas_int n, const float *x, rocblas_int incx, rocblas_int *result)

rocblas_status rocblas_idamin(rocblas_handle handle, rocblas_int n, const double *x, rocblas_int incx, rocblas_int *result)

rocblas_status rocblas_icamin(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *x, rocblas_int incx, rocblas_int *result)

rocblas_status rocblas_izamin(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *x, rocblas_int incx, rocblas_int *result)

rocblas_status rocblas_isamin_batched(rocblas_handle handle, rocblas_int n, const float *const x[], rocblas_int incx, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_idamin_batched(rocblas_handle handle, rocblas_int n, const double *const x[], rocblas_int incx, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_icamin_batched(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *const x[], rocblas_int incx, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_izamin_batched(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *const x[], rocblas_int incx, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_isamin_strided_batched(rocblas_handle handle, rocblas_int n, const float *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_idamin_strided_batched(rocblas_handle handle, rocblas_int n, const double *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_icamin_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, rocblas_int *result)

rocblas_status rocblas_izamin_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, rocblas_int *result)

5.4.3 asum 函数族

asum 计算实数向量 x 元素的幅值之和，或者如果 x 是复数向量，则计算元素实部与虚部的幅值之和。

参数：

1) handle：上下文队列的句柄；

2) n：向量 x 和 y 中的元素数量；

3) x：存储向量 x 设备指针；

4) incx：指定 x 元素的增量；

5) result：存储结果；

6) batch_count：批次数；

7) stridex：x_i 与 x_(i+1)之间的步长；

rocblas_status rocblas_sasum(rocblas_handle handle, rocblas_int n, const float *x, roc blas_int incx, float *result)

rocblas_status rocblas_dasum(rocblas_handle handle, rocblas_int n, const double *x, r ocblas_int incx, double *result)

rocblas_status rocblas_scasum(rocblas_handle handle, rocblas_int n, const rocblas_flo at_complex *x, rocblas_int incx, float *result)

rocblas_status rocblas_dzasum(rocblas_handle handle, rocblas_int n, const rocblas_do uble_complex *x, rocblas_int incx, double *result)

rocblas_status rocblas_sasum_batched(rocblas_handle handle, rocblas_int n, const flo at *const x[], rocblas_int incx, rocblas_int batch_count, float *results)

rocblas_status rocblas_dasum_batched(rocblas_handle handle, rocblas_int n, const do uble *const x[], rocblas_int incx, rocblas_int batch_count, double *results)

rocblas_status rocblas_dzasum_batched(rocblas_handle handle, rocblas_int n, const r ocblas_double_complex *const x[], rocblas_int incx, rocblas_int batch_count, double *results)

rocblas_status rocblas_sasum_strided_batched(rocblas_handle handle, rocblas_int n, c onst float *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, float *r esults)

rocblas_status rocblas_dasum_strided_batched(rocblas_handle handle, rocblas_int n, const double *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, dou ble *results)

rocblas_status rocblas_scasum_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_int b

atch_count, float *results)

rocblas_status rocblas_dzasum_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, double *results)

5.4.4 axpy 函数族

axpy 计算常数 alpha 乘以向量 x，再加上向量 y。

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 和 y 中的元素数量；
3) alpha：标量 alpha；
4) x：存储向量 x 设备指针；
5) incx：指定 x 元素的增量；
6) y：存储向量 y 设备指针；
7) incy：指定 y 元素的增量；
8) batch_count：批次数；
9) stridex：x 向量之间的步长；
10) stridey：y 向量之间的步长；
11) result：存储结果；

rocblas_status rocblas_saxpy(rocblas_handle handle, rocblas_int n, const float *alpha, const float *x, rocblas_int incx, float *y, rocblas_int incy)

rocblas_status rocblas_daxpy(rocblas_handle handle, rocblas_int n, const double *alpha, const double *x, rocblas_int incx, double *y, rocblas_int incy)

rocblas_status rocblas_haxpy(rocblas_handle handle, rocblas_int n, const rocblas_half *alpha, const rocblas_half *x, rocblas_int incx, rocblas_half *y, rocblas_int incy)

rocblas_status rocblas_caxpy(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zaxpy(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int incx, rocblas_dou

开发者社区

ble_complex *y, rocblas_int incy)

rocblas_status rocblas_saxpy_batched(rocblas_handle handle, rocblas_int n, const flo at *alpha, const float *const x[], rocblas_int incx, float *const y[], rocblas_int incy, ro cblas_int batch_count)

rocblas_status rocblas_daxpy_batched(rocblas_handle handle, rocblas_int n, const do uble *alpha, const double *const x[], rocblas_int incx, double *const y[], rocblas_int i ncy, rocblas_int batch_count)

rocblas_status rocblas_haxpy_batched(rocblas_handle handle, rocblas_int n, const roc blas_half *alpha, const rocblas_half *const x[], rocblas_int incx, rocblas_half *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_caxpy_batched(rocblas_handle handle, rocblas_int n, const roc blas_float_complex *alpha, const rocblas_float_complex *const x[], rocblas_int incx, rocblas_float_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_zaxpy_batched(rocblas_handle handle, rocblas_int n, const roc blas_double_complex *alpha, const rocblas_double_complex *const x[], rocblas_int i ncx, rocblas_double_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_saxpy_strided_batched(rocblas_handle handle, rocblas_int n, c onst float *alpha, const float *x, rocblas_int incx, rocblas_stride stridex, float *y, rocb las_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_daxpy_strided_batched(rocblas_handle handle, rocblas_int n, c onst double *alpha, const double *x, rocblas_int incx, rocblas_stride stridex, double * y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_haxpy_strided_batched(rocblas_handle handle, rocblas_int n, c onst rocblas_half *alpha, const rocblas_half *x, rocblas_int incx, rocblas_stride stride x, rocblas_half *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_caxpy_strided_batched(rocblas_handle handle, rocblas_int n, c onst rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_float_complex *y, rocblas_int incy, rocblas_stride stri dey, rocblas_int batch_count)

rocblas_status rocblas_zaxpy_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_double_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

5.4.5 copy 函数族

copy 将每个元素 x[i] 复制到 y[i] 中，其中 i = 1 , ... , n。

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 和 y 中的元素数量；
3) x：存储向量 x 设备指针；
4) incx：指定 x 元素的增量；
5) y：存储向量 y 设备指针；
6) incy：指定 y 元素的增量；
7) batch_count：批次数；
8) stridex：x 向量之间的步长；
9) stridey：y 向量之间的步长；
10) result：存储结果；

rocblas_status rocblas_scopy(rocblas_handle handle, rocblas_int n, const float *x, rocblas_int incx, float *y, rocblas_int incy)

rocblas_status rocblas_dcopy(rocblas_handle handle, rocblas_int n, const double *x, rocblas_int incx, double *y, rocblas_int incy)

rocblas_status rocblas_ccopy(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *x, rocblas_int incx, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zcopy(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *x, rocblas_int incx, rocblas_double_complex *y, rocblas_int incy)

rocblas_status rocblas_scopy_batched(rocblas_handle handle, rocblas_int n, const float *const x[], rocblas_int incx, float *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_dcopy_batched(rocblas_handle handle, rocblas_int n, const do

开发者社区

uble *const x[], rocblas_int incx, double *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_ccopy_batched(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *const x[], rocblas_int incx, rocblas_float_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_zcopy_batched(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *const x[], rocblas_int incx, rocblas_double_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_scopy_strided_batched(rocblas_handle handle, rocblas_int n, const float *x, rocblas_int incx, rocblas_stride stridex, float *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_dcopy_strided_batched(rocblas_handle handle, rocblas_int n, const double *x, rocblas_int incx, rocblas_stride stridex, double *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_ccopy_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_float_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_zcopy_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_double_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

5.4.6 dot 函数族

dot(u) 计算向量 x 和 y 的点积。

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 和 y 中的元素数量；
3) x：存储向量 x 设备指针；
4) incx：指定 x 元素的增量；
5) y：存储向量 y 设备指针；
6) incy：指定 y 元素的增量；

7) batch_count：批次数；

8) stridex：x 向量之间的步长；

9) stridey：y 向量之间的步长；

10) result：存储结果；

rocblas_status rocblas_sdot(rocblas_handle handle, rocblas_int n, const float *x, rocblas_int incx, const float *y, rocblas_int incy, float *result)

rocblas_status rocblas_ddot(rocblas_handle handle, rocblas_int n, const double *x, rocblas_int incx, const double *y, rocblas_int incy, double *result)

rocblas_status rocblas_hdot(rocblas_handle handle, rocblas_int n, const rocblas_half *x, rocblas_int incx, const rocblas_half *y, rocblas_int incy, rocblas_half *result)

rocblas_status rocblas_bfdot(rocblas_handle handle, rocblas_int n, const rocblas_bfloat16 *x, rocblas_int incx, const rocblas_bfloat16 *y, rocblas_int incy, rocblas_bfloat16 *result)

rocblas_status rocblas_cdotu(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *y, rocblas_int incy, rocblas_float_complex *result)

rocblas_status rocblas_cdotc(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *y, rocblas_int incy, rocblas_float_complex *result)

rocblas_status rocblas_zdotu(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *y, rocblas_int incy, rocblas_double_complex *result)

rocblas_status rocblas_zdotc(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *y, rocblas_int incy, rocblas_double_complex *result)

rocblas_status rocblas_sdot_batched(rocblas_handle handle, rocblas_int n, const float *const x[], rocblas_int incx, const float *const y[], rocblas_int incy, rocblas_int batch_count, float *result)

rocblas_status rocblas_ddot_batched(rocblas_handle handle, rocblas_int n, const doub

le *const x[], rocblas_int incx, const double *const y[], rocblas_int incy, rocblas_int b
atch_count, double *result)

rocblas_status rocblas_hdot_batched(rocblas_handle handle, rocblas_int n, const rocbl
as_half *const x[], rocblas_int incx, const rocblas_half *const y[], rocblas_int incy, ro
cblas_int batch_count, rocblas_half *result)

rocblas_status rocblas_bfdot_batched(rocblas_handle handle, rocblas_int n, const roc
blas_bfloat16 *const x[], rocblas_int incx, const rocblas_bfloat16 *const y[], rocblas_
int incy, rocblas_int batch_count, rocblas_bfloat16 *result)

rocblas_status rocblas_cdotu_batched(rocblas_handle handle, rocblas_int n, const roc
blas_float_complex *const x[], rocblas_int incx, const rocblas_float_complex *const
y[], rocblas_int incy, rocblas_int batch_count, rocblas_float_complex *result)

rocblas_status rocblas_cdotc_batched(rocblas_handle handle, rocblas_int n, const roc
blas_float_complex *const x[], rocblas_int incx, const rocblas_float_complex *const
y[], rocblas_int incy, rocblas_int batch_count, rocblas_float_complex *result)

rocblas_status rocblas_zdotu_batched(rocblas_handle handle, rocblas_int n, const roc
blas_double_complex *const x[], rocblas_int incx, const rocblas_double_complex *co
nst y[], rocblas_int incy, rocblas_int batch_count, rocblas_double_complex *result)

rocblas_status rocblas_zdotc_batched(rocblas_handle handle, rocblas_int n, const roc
blas_double_complex *const x[], rocblas_int incx, const rocblas_double_complex *co
nst y[], rocblas_int incy, rocblas_int batch_count, rocblas_double_complex *result)

rocblas_status rocblas_sdot_strided_batched(rocblas_handle handle, rocblas_int n, co
nst float *x, rocblas_int incx, rocblas_stride stridex, const float *y, rocblas_int incy, ro
cblas_stride stridey, rocblas_int batch_count, float *result)

rocblas_status rocblas_ddot_strided_batched(rocblas_handle handle, rocblas_int n, co
nst double *x, rocblas_int incx, rocblas_stride stridex, const double *y, rocblas_int inc
y, rocblas_stride stridey, rocblas_int batch_count, double *result)

rocblas_status rocblas_hdot_strided_batched(rocblas_handle handle, rocblas_int n, co
nst rocblas_half *x, rocblas_int incx, rocblas_stride stridex, const rocblas_half *y, roc
blas_int incy, rocblas_stride stridey, rocblas_int batch_count, rocblas_half *result)

rocblas_status rocblas_bfdot_strided_batched(rocblas_handle handle, rocblas_int n, c
onst rocblas_bfloat16 *x, rocblas_int incx, rocblas_stride stridex, const rocblas_bfloat
16 *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count, rocblas_bfloat
16 *result)

rocblas_status rocblas_cdotu_strided_batched(rocblas_handle handle, rocblas_int n, c
onst rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, const rocblas_
float_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count, ro
cblas_float_complex *result)

rocblas_status rocblas_cdotc_strided_batched(rocblas_handle handle, rocblas_int n, c
onst rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, const rocblas_
float_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count, ro
cblas_float_complex *result)

rocblas_status rocblas_zdotu_strided_batched(rocblas_handle handle, rocblas_int n, c
onst rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, const rocbla
s_double_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_coun
t, rocblas_double_complex *result)

rocblas_status rocblas_zdotc_strided_batched(rocblas_handle handle, rocblas_int n, c
onst rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, const rocbla
s_double_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_coun
t, rocblas_double_complex *result)

5.4.7 nrm2 函数族

nrm2 计算实数或复数向量的欧几里得范数。

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 和 y 中的元素数量；
3) x：存储向量 x 设备指针；
4) incx：指定 x 元素的增量；
5) batch_count：批次数；
6) stridex：x 向量之间的步长；

7) result：存储结果；

rocblas_status rocblas_snrm2(rocblas_handle handle, rocblas_int n, const float *x, roc
blas_int incx, float *result)

rocblas_status rocblas_dnrm2(rocblas_handle handle, rocblas_int n, const double *x, r
ocblas_int incx, double *result)

rocblas_status rocblas_scnrm2(rocblas_handle handle, rocblas_int n, const rocblas_flo
at_complex *x, rocblas_int incx, float *result)

rocblas_status rocblas_dznrm2(rocblas_handle handle, rocblas_int n, const rocblas_do
uble_complex *x, rocblas_int incx, double *result)

rocblas_status rocblas_snrm2_batched(rocblas_handle handle, rocblas_int n, const flo
at *const x[], rocblas_int incx, rocblas_int batch_count, float *results)

rocblas_status rocblas_dnrm2_batched(rocblas_handle handle, rocblas_int n, const do
uble *const x[], rocblas_int incx, rocblas_int batch_count, double *results)

rocblas_status rocblas_scnrm2_batched(rocblas_handle handle, rocblas_int n, const ro
cblas_float_complex *const x[], rocblas_int incx, rocblas_int batch_count, float *resu
lts)

rocblas_status rocblas_dznrm2_batched(rocblas_handle handle, rocblas_int n, const r
ocblas_double_complex *const x[], rocblas_int incx, rocblas_int batch_count, double
*results)

rocblas_status rocblas_snrm2_strided_batched(rocblas_handle handle, rocblas_int n, c
onst float *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, float *r
esults)

rocblas_status rocblas_dnrm2_strided_batched(rocblas_handle handle, rocblas_int n,
const double *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, dou
ble *results)

rocblas_status rocblas_scnrm2_strided_batched(rocblas_handle handle, rocblas_int n,
const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_int b
atch_count, float *results)

rocblas_status rocblas_dznrm2_strided_batched(rocblas_handle handle, rocblas_int n,

const rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_int batch_count, double *results)

5.4.8 rot 函数族

rot 将由 c=cos(alpha) 和 s=sin(alpha) 定义的 Givens 旋转矩阵应用于向量 x 和 y。标量 c 和 s 可以存储在主机或设备内存中,位置通过调用 rocblas_set_pointer_mode 指定。

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 和 y 中的元素数量；
3) x：存储向量 x 设备指针；
4) incx：指定 x 元素的增量；
5) y：存储向量 y 设备指针；
6) incy：指定 y 元素的增量；
7) c：设备指针或主机指针存储旋转矩阵的标量余弦分量；
8) s：设备指针或主机指针存储旋转矩阵的标量正弦分量；
9) batch_count：批次数；
10) stridex：x 向量之间的步长；
11) stridey：y 向量之间的步长；
12) result：存储结果；

rocblas_status rocblas_srot(rocblas_handle handle, rocblas_int n, float *x, rocblas_int incx, float *y, rocblas_int incy, const float *c, const float *s)

rocblas_status rocblas_drot(rocblas_handle handle, rocblas_int n, double *x, rocblas_int incx, double *y, rocblas_int incy, const double *c, const double *s)

rocblas_status rocblas_crot(rocblas_handle handle, rocblas_int n, rocblas_float_complex *x, rocblas_int incx, rocblas_float_complex *y, rocblas_int incy, const float *c, const rocblas_float_complex *s)

rocblas_status rocblas_csrot(rocblas_handle handle, rocblas_int n, rocblas_float_complex *x, rocblas_int incx, rocblas_float_complex *y, rocblas_int incy, const float *c, const float *s)

rocblas_status rocblas_zrot(rocblas_handle handle, rocblas_int n, rocblas_double_complex *x, rocblas_int incx, rocblas_double_complex *y, rocblas_int incy, const double *c, const rocblas_double_complex *s)

rocblas_status rocblas_zdrot(rocblas_handle handle, rocblas_int n, rocblas_double_complex *x, rocblas_int incx, rocblas_double_complex *y, rocblas_int incy, const double *c, const double *s)

rocblas_status rocblas_srot_batched(rocblas_handle handle, rocblas_int n, float *const x[], rocblas_int incx, float *const y[], rocblas_int incy, const float *c, const float *s, rocblas_int batch_count)

rocblas_status rocblas_drot_batched(rocblas_handle handle, rocblas_int n, double *const x[], rocblas_int incx, double *const y[], rocblas_int incy, const double *c, const double *s, rocblas_int batch_count)

rocblas_status rocblas_crot_batched(rocblas_handle handle, rocblas_int n, rocblas_float_complex *const x[], rocblas_int incx, rocblas_float_complex *const y[], rocblas_int incy, const float *c, const rocblas_float_complex *s, rocblas_int batch_count)

rocblas_status rocblas_csrot_batched(rocblas_handle handle, rocblas_int n, rocblas_float_complex *const x[], rocblas_int incx, rocblas_float_complex *const y[], rocblas_int incy, const float *c, const float *s, rocblas_int batch_count)

rocblas_status rocblas_zrot_batched(rocblas_handle handle, rocblas_int n, rocblas_double_complex *const x[], rocblas_int incx, rocblas_double_complex *const y[], rocblas_int incy, const double *c, const rocblas_double_complex *s, rocblas_int batch_count)

rocblas_status rocblas_zdrot_batched(rocblas_handle handle, rocblas_int n, rocblas_double_complex *const x[], rocblas_int incx, rocblas_double_complex *const y[], rocblas_int incy, const double *c, const double *s, rocblas_int batch_count)

rocblas_status rocblas_srot_strided_batched(rocblas_handle handle, rocblas_int n, float *x, rocblas_int incx, rocblas_stride stride_x, float *y, rocblas_int incy, rocblas_stride stride_y, const float *c, const float *s, rocblas_int batch_count)

rocblas_status rocblas_drot_strided_batched(rocblas_handle handle, rocblas_int n, do

uble *x, rocblas_int incx, rocblas_stride stride_x, double *y, rocblas_int incy, rocblas_stride stride_y, const double *c, const double *s, rocblas_int batch_count)

rocblas_status rocblas_crot_strided_batched(rocblas_handle handle, rocblas_int n, rocblas_float_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_float_complex *y, rocblas_int incy, rocblas_stride stride_y, const float *c, const rocblas_float_complex *s, rocblas_int batch_count)

rocblas_status rocblas_csrot_strided_batched(rocblas_handle handle, rocblas_int n, rocblas_float_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_float_complex *y, rocblas_int incy, rocblas_stride stride_y, const float *c, const float *s, rocblas_int batch_count)

rocblas_status rocblas_zrot_strided_batched(rocblas_handle handle, rocblas_int n, rocblas_double_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_double_complex *y, rocblas_int incy, rocblas_stride stride_y, const double *c, const rocblas_double_complex *s, rocblas_int batch_count)

rocblas_status rocblas_zdrot_strided_batched(rocblas_handle handle, rocblas_int n, rocblas_double_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_double_complex *y, rocblas_int incy, rocblas_stride stride_y, const double *c, const double *s, rocblas_int batch_count)

5.4.9 rotg 函数族

rotg 为向量 (ab) 创建 Givens 旋转矩阵。标量 a、b、c 和 s 可以存储在主机或设备内存中，位置通过调用 rocblas_set_pointer_mode 指定。

参数：

1) handle：上下文队列的句柄；
2) a：指向 a 的指针，向量 (a,b) 中的元素，用 r 覆盖；
3) b：指向 b 的指针，向量 (a,b) 中的元素，用 z 覆盖；
4) c：指向 c 的指针，Givens 旋转的余弦元素；
5) s：指向 s 的[out]指针，Givens 旋转的正弦元素；
6) batch_count：批次数；
7) stride_a：批次中 a 元素之间的距离；

8) stride_b：批次中 b 元素之间的距离；

9) stride_c：批次中 c 元素之间的距离；

10) stride_s：批次中 s 元素之间的距离；

rocblas_status rocblas_srotg(rocblas_handle handle, float *a, float *b, float *c, float *s)

rocblas_status rocblas_drotg(rocblas_handle handle, double *a, double *b, double *c, double *s)

rocblas_status rocblas_crotg(rocblas_handle handle, rocblas_float_complex *a, rocblas_float_complex *b, float *c, rocblas_float_complex *s)

rocblas_status rocblas_zrotg(rocblas_handle handle, rocblas_double_complex *a, rocblas_double_complex *b, double *c, rocblas_double_complex *s)

rocblas_status rocblas_srotg_batched(rocblas_handle handle, float *const a[], float *const b[], float *const c[], float *const s[], rocblas_int batch_count)

rocblas_status rocblas_drotg_batched(rocblas_handle handle, double *const a[], double *const b[], double *const c[], double *const s[], rocblas_int batch_count)

rocblas_status rocblas_crotg_batched(rocblas_handle handle, rocblas_float_complex *const a[], rocblas_float_complex *const b[], float *const c[], rocblas_float_complex *const s[], rocblas_int batch_count)

rocblas_status rocblas_zrotg_batched(rocblas_handle handle, rocblas_double_complex *const a[], rocblas_double_complex *const b[], double *const c[], rocblas_double_complex *const s[], rocblas_int batch_count)

rocblas_status rocblas_srotg_strided_batched(rocblas_handle handle, float *a, rocblas_stride stride_a, float *b, rocblas_stride stride_b, float *c, rocblas_stride stride_c, float *s, rocblas_stride stride_s, rocblas_int batch_count)

rocblas_status rocblas_drotg_strided_batched(rocblas_handle handle, double *a, rocblas_stride stride_a, double *b, rocblas_stride stride_b, double *c, rocblas_stride stride_c, double *s, rocblas_stride stride_s, rocblas_int batch_count)

rocblas_status rocblas_crotg_strided_batched(rocblas_handle handle, rocblas_float_complex *a, rocblas_stride stride_a, rocblas_float_complex *b, rocblas_stride stride_b,

float *c, rocblas_stride stride_c, rocblas_float_complex *s, rocblas_stride stride_s, rocblas_int batch_count)

rocblas_status rocblas_zrotg_strided_batched(rocblas_handle handle, rocblas_double_complex *a, rocblas_stride stride_a, rocblas_double_complex *b, rocblas_stride stride_b, double *c, rocblas_stride stride_c, rocblas_double_complex *s, rocblas_stride stride_s, rocblas_int batch_count)

5.4.10 rotm 函数族

rotm 将 param 定义的修改后的 Givens 旋转矩阵应用于向量 x 和 y。

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 和 y 中的元素数量；
3) x：存储向量 x 设备指针；
4) incx：指定 x 元素的增量；
5) y：存储向量 y 设备指针；
6) incy：指定 y 元素的增量；
7) param：定义旋转的 5 个元素的设备向量或主机向量；
8) stride_x：y 向量之间的步长；
9) stride_y：y 向量之间的步长；

rocblas_status rocblas_srotm(rocblas_handle handle, rocblas_int n, float *x, rocblas_int incx, float *y, rocblas_int incy, const float *param)

rocblas_status rocblas_drotm(rocblas_handle handle, rocblas_int n, double *x, rocblas_int incx, double *y, rocblas_int incy, const double *param)

rocblas_status rocblas_srotm_batched(rocblas_handle handle, rocblas_int n, float *const x[], rocblas_int incx, float *const y[], rocblas_int incy, const float *const param[], rocblas_int batch_count)

rocblas_status rocblas_drotm_batched(rocblas_handle handle, rocblas_int n, double *const x[], rocblas_int incx, double *const y[], rocblas_int incy, const double *const param[], rocblas_int batch_count)

rocblas_status rocblas_srotm_strided_batched(rocblas_handle handle, rocblas_int n, fl

oat *x, rocblas_int incx, rocblas_stride stride_x, float *y, rocblas_int incy, rocblas_stride stride_y, const float *param, rocblas_stride stride_param, rocblas_int batch_count)

rocblas_status rocblas_drotm_strided_batched(rocblas_handle handle, rocblas_int n, double *x, rocblas_int incx, rocblas_stride stride_x, double *y, rocblas_int incy, rocblas_stride stride_y, const double *param, rocblas_stride stride_param, rocblas_int batch_count)

5.4.11 rotmg 函数族

rotmg 为向量 (d1 * x1, d2 * y1) 创建修改后的 Givens 旋转矩阵。参数可以存储在主机或设备内存中。位置通过调用 rocblas_set_pointer_mode 指定。

参数：

1) handle：rocblas 库上下文队列的句柄；
2) d1：指向被覆盖的输入标量的设备指针或主机指针；
3) d2：指向被覆盖的输入标量的设备指针或主机指针；
4) x1：指向被覆盖的输入标量的设备指针或主机指针；
5) y1：指向输入标量的设备指针或主机指针；
6) param：定义旋转的五个元素的设备向量或主机向量；
7) stride_d1：指定 d1_i 的开头和 d1_(i+1) 之间的增量；
8) stride_d2：指定 d2_i 的开头和 d2_(i+1) 之间的增量；
9) stride_x1：指定 x1_i 的开头和 x1_(i+1) 之间的增量；
10) stride_y1：指定 y1_i 的开头和 y1_(i+1) 之间的增量；
11) stride_param：指定 param_i 的开头和 param_(i + 1) 之间的增量；
12) batch_count：批次数；

rocblas_status rocblas_srotmg(rocblas_handle handle, float *d1, float *d2, float *x1, const float *y1, float *param)

rocblas_status rocblas_drotmg(rocblas_handle handle, double *d1, double *d2, double *x1, const double *y1, double *param)

rocblas_status rocblas_srotmg_batched(rocblas_handle handle, float *const d1[], float *const d2[], float *const x1[], const float *const y1[], float *const param[], rocblas_int batch_count)

rocblas_status rocblas_drotmg_batched(rocblas_handle handle, double *const d1[], double *const d2[], double *const x1[], const double *const y1[], double *const param[], rocblas_int batch_count)

rocblas_status rocblas_srotmg_strided_batched(rocblas_handle handle, float *d1, rocblas_stride stride_d1, float *d2, rocblas_stride stride_d2, float *x1, rocblas_stride stride_x1, const float *y1, rocblas_stride stride_y1, float *param, rocblas_stride stride_param, rocblas_int batch_count)

rocblas_status rocblas_drotmg_strided_batched(rocblas_handle handle, double *d1, rocblas_stride stride_d1, double *d2, rocblas_stride stride_d2, double *x1, rocblas_stride stride_x1, const double *y1, rocblas_stride stride_y1, double *param, rocblas_stride stride_param, rocblas_int batch_count)

5.4.12 scal 函数族

scal 使用标量 alpha 缩放向量 x 的每个元素。

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 中的元素数量；
3) alpha：标量 alpha 的设备指针或主机指针；
4) x：存储向量 x 设备指针；
5) incx：指定 x 元素的增量；
6) batch_count：批次数；
7) stride_x：一个向量 (x_i) 的起始位置到下一个向量 (x_i+1) 的步长；

rocblas_status rocblas_sscal(rocblas_handle handle, rocblas_int n, const float *alpha, float *x, rocblas_int incx)

rocblas_status rocblas_dscal(rocblas_handle handle, rocblas_int n, const double *alpha, double *x, rocblas_int incx)

rocblas_status rocblas_cscal(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *alpha, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_zscal(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *alpha, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_csscal(rocblas_handle handle, rocblas_int n, const float *alpha, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_zdscal(rocblas_handle handle, rocblas_int n, const double *alpha, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_sscal_batched(rocblas_handle handle, rocblas_int n, const float *alpha, float *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_dscal_batched(rocblas_handle handle, rocblas_int n, const double *alpha, double *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_cscal_batched(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *alpha, rocblas_float_complex *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_zscal_batched(rocblas_handle handle, rocblas_int n, const rocblas_double_complex *alpha, rocblas_double_complex *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_csscal_batched(rocblas_handle handle, rocblas_int n, const float *alpha, rocblas_float_complex *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_zdscal_batched(rocblas_handle handle, rocblas_int n, const double *alpha, rocblas_double_complex *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_sscal_strided_batched(rocblas_handle handle, rocblas_int n, const float *alpha, float *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_dscal_strided_batched(rocblas_handle handle, rocblas_int n, const double *alpha, double *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_cscal_strided_batched(rocblas_handle handle, rocblas_int n, const rocblas_float_complex *alpha, rocblas_float_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_zscal_strided_batched(rocblas_handle handle, rocblas_int n, co

nst rocblas_double_complex *alpha, rocblas_double_complex *x, rocblas_int incx, ro
cblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_csscal_strided_batched(rocblas_handle handle, rocblas_int n, c
onst float *alpha, rocblas_float_complex *x, rocblas_int incx, rocblas_stride stride_x,
rocblas_int batch_count)

rocblas_status rocblas_zdscal_strided_batched(rocblas_handle handle, rocblas_int n, c
onst double *alpha, rocblas_double_complex *x, rocblas_int incx, rocblas_stride strid
e_x, rocblas_int batch_count)

5.4.13 swap 函数族

交换向量 x 和 y。

参数：

1) handle：上下文队列的句柄；
2) n：向量 x 中的元素数量；
3) x：存储向量 x 设备指针；
4) incx：指定 x 元素的增量；
5) y：存储向量 y 设备指针；
6) incy：指定 y 元素的增量；
7) batch_count：批次数；
8) stridex：一个向量 (x_i) 的起始位置到下一个向量 (x_i+1) 的步长；
9) stridey：一个向量 (y_i) 的起始位置到下一个向量 (y_i+1) 的步长；

rocblas_status rocblas_sswap(rocblas_handle handle, rocblas_int n, float *x, rocblas_i
nt incx, float *y, rocblas_int incy)

rocblas_status rocblas_dswap(rocblas_handle handle, rocblas_int n, double *x, rocbla
s_int incx, double *y, rocblas_int incy)

rocblas_status rocblas_cswap(rocblas_handle handle, rocblas_int n, rocblas_float_co
mplex *x, rocblas_int incx, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zswap(rocblas_handle handle, rocblas_int n, rocblas_double_c
omplex *x, rocblas_int incx, rocblas_double_complex *y, rocblas_int incy)

rocblas_status rocblas_sswap_batched(rocblas_handle handle, rocblas_int n, float *co

nst x[], rocblas_int incx, float *const y[], rocblas_int incy, rocblas_int batch_count)
rocblas_status rocblas_dswap_batched(rocblas_handle handle, rocblas_int n, double *const x[], rocblas_int incx, double *const y[], rocblas_int incy, rocblas_int batch_count)
rocblas_status rocblas_cswap_batched(rocblas_handle handle, rocblas_int n, rocblas_float_complex *const x[], rocblas_int incx, rocblas_float_complex *const y[], rocblas_int incy, rocblas_int batch_count)
rocblas_status rocblas_zswap_batched(rocblas_handle handle, rocblas_int n, rocblas_double_complex *const x[], rocblas_int incx, rocblas_double_complex *const y[], rocblas_int incy, rocblas_int batch_count)
rocblas_status rocblas_sswap_strided_batched(rocblas_handle handle, rocblas_int n, float *x, rocblas_int incx, rocblas_stride stridex, float *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)
rocblas_status rocblas_dswap_strided_batched(rocblas_handle handle, rocblas_int n, double *x, rocblas_int incx, rocblas_stride stridex, double *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)
rocblas_status rocblas_cswap_strided_batched(rocblas_handle handle, rocblas_int n, rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_float_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)
rocblas_status rocblas_zswap_strided_batched(rocblas_handle handle, rocblas_int n, rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_double_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

5.5 rocBLAS 二级函数

rocBLAS 二级函数执行矩阵向量运算。

5.5.1 gbmv 函数族

gbmv 执行带状矩阵向量运算。

参数：

1) handle：上下文队列的句柄；

2) trans：表示矩阵 A 是否被转置（共轭）；

3) m：矩阵 A 的行数；

4) n：矩阵 A 的列数；

5) kl：A 的子对角线的数量；

6) ku：A 的超对角线的数量；

7) alpha：指向标量 alpha 的设备指针或主机指针；

8) A：设备指针，用于存储带状矩阵 A；

9) lda：指定 A 的主维度；

10) x：存储向量 x 的设备指针；

11) incx：指定 x 元素的增量；

12) beta：指向标量 beta 的设备指针或主机指针；

13) y：存储向量 y 的设备指针；

14) incy：指定 y 元素的增量；

15) batch_count：批次数；

16) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；

17) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

18) stride_y：一个向量（y_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_sgbmv(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const float *alpha, const float *A, rocblas_int lda, const float *x, rocblas_int incx, const float *beta, float *y, rocblas_int incy)

rocblas_status rocblas_dgbmv(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const double *alpha, const double *A, rocblas_int lda, const double *x, rocblas_int incx, const double *beta, double *y, rocblas_int incy)

rocblas_status rocblas_cgbmv(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *y, rocb

开发者社区

las_int incy)

rocblas_status rocblas_zgbmv(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *y, rocblas_int incy)

rocblas_status rocblas_sgbmv_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const float *alpha, const float *const A[], rocblas_int lda, const float *const x[], rocblas_int incx, const float *beta, float *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_dgbmv_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const double *alpha, const double *const A[], rocblas_int lda, const double *const x[], rocblas_int incx, const double *beta, double *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_cgbmv_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const x[], rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_zgbmv_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const x[], rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_sgbmv_strided_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const float *alpha, const float *A, rocblas_int lda, rocblas_stride stride_A, const float *x, rocblas_int incx, rocblas_stride stride_x, const float *beta, float *y, rocblas_int incy, rocblas_stride stride_y, rocblas_int batch_count)

rocblas_status rocblas_dgbmv_strided_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const double *alpha, const double *A, rocblas_int lda, rocblas_stride stride_A, const double *x, rocblas_int incx, rocblas_stride stride_x, const double *beta, double *y, rocblas_int incy, rocblas_stride stride_y, rocblas_int batch_count)

rocblas_status rocblas_cgbmv_strided_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stride_x, const rocblas_float_complex *beta, rocblas_float_complex *y, rocblas_int incy, rocblas_stride stride_y, rocblas_int batch_count)

rocblas_status rocblas_zgbmv_strided_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, rocblas_int kl, rocblas_int ku, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *x, rocblas_int incx, rocblas_stride stride_x, const rocblas_double_complex *beta, rocblas_double_complex *y, rocblas_int incy, rocblas_stride stride_y, rocblas_int batch_count)

5.5.2 gemv 函数族

gemv 执行通用矩阵向量运算。

参数：

1) handle：上下文队列的句柄；
2) trans：表示矩阵 A 是否被转置（共轭）；
3) m：矩阵 A 的行数；
4) n：矩阵 A 的列数；
5) alpha：指向标量 alpha 的设备指针或主机指针；
6) A：设备指针，存储矩阵 A；
7) lda：指定 A 的主维度；
8) x：存储向量 x 的设备指针；
9) incx：指定 x 元素的增量；

10) beta：指向标量 beta 的设备指针或主机指针；

11) y：存储向量 y 的设备指针；

12) incy：指定 y 元素的增量；

13) batch_count：批次数；

14) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；

15) stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

16) stridey：一个向量（y_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_sgemv(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, const float *x, rocblas_int incx, const float *beta, float *y, rocblas_int incy)

rocblas_status rocblas_dgemv(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, const double *x, rocblas_int incx, const double *beta, double *y, rocblas_int incy)

rocblas_status rocblas_cgemv(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zgemv(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *y, rocblas_int incy)

rocblas_status rocblas_sgemv_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, const float *alpha, const float *const A[], rocblas_int lda, const float *const x[], rocblas_int incx, const float *beta, float *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_dgemv_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, const double *alpha, const double *const A[], rocblas_int lda, const double *const x[], rocblas_int incx, const double *beta, double *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_cgemv_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const x[], rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_zgemv_batched(rocblas_handle handle, rocblas_operation trans, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const x[], rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_sgemv_strided_batched(rocblas_handle handle, rocblas_operation transA, rocblas_int m, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, rocblas_stride strideA, const float *x, rocblas_int incx, rocblas_stride stridex, const float *beta, float *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_dgemv_strided_batched(rocblas_handle handle, rocblas_operation transA, rocblas_int m, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, rocblas_stride strideA, const double *x, rocblas_int incx, rocblas_stride stridex, const double *beta, double *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_cgemv_strided_batched(rocblas_handle handle, rocblas_operation transA, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride strideA, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, const rocblas_float_complex *beta, rocblas_float_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_zgemv_strided_batched(rocblas_handle handle, rocblas_operation transA, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride strideA, const rocblas_do

uble_complex *x, rocblas_int incx, rocblas_stride stridex, const rocblas_double_complex *beta, rocblas_double_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

5.5.3 ger 函数族

ger / geru / gerc 是用来执行矩阵的 rank-1 外积更新操作，即将两个向量的乘积加到矩阵上，用于高效构造或更新矩阵。

参数：

1) handle：上下文队列的句柄；
2) m：矩阵 A 的行数；
3) n：矩阵 A 的列数；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) A：设备指针，存储矩阵 A；
6) lda：指定 A 的主维度；
7) x：存储向量 x 的设备指针；
8) incx：指定 x 元素的增量；
9) y：存储向量 y 的设备指针；
10) incy：指定 y 元素的增量；
11) batch_count：批次数；
12) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
13) stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；
14) stridey：一个向量（y_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_sger(rocblas_handle handle, rocblas_int m, rocblas_int n, const float *alpha, const float *x, rocblas_int incx, const float *y, rocblas_int incy, float *A, rocblas_int lda)

rocblas_status rocblas_dger(rocblas_handle handle, rocblas_int m, rocblas_int n, const double *alpha, const double *x, rocblas_int incx, const double *y, rocblas_int incy, double *A, rocblas_int lda)

rocblas_status rocblas_cgeru(rocblas_handle handle, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, c

开发者社区

onst rocblas_float_complex *y, rocblas_int incy, rocblas_float_complex *A, rocblas_int lda)

rocblas_status rocblas_zgeru(rocblas_handle handle, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *y, rocblas_int incy, rocblas_double_complex *A, rocblas_int lda)

rocblas_status rocblas_cgerc(rocblas_handle handle, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *y, rocblas_int incy, rocblas_float_complex *A, rocblas_int lda)

rocblas_status rocblas_zgerc(rocblas_handle handle, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *y, rocblas_int incy, rocblas_double_complex *A, rocblas_int lda)

rocblas_status rocblas_sger_batched(rocblas_handle handle, rocblas_int m, rocblas_int n, const float *alpha, const float *const x[], rocblas_int incx, const float *const y[], rocblas_int incy, float *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_dger_batched(rocblas_handle handle, rocblas_int m, rocblas_int n, const double *alpha, const double *const x[], rocblas_int incx, const double *const y[], rocblas_int incy, double *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_cgeru_batched(rocblas_handle handle, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const x[], rocblas_int incx, const rocblas_float_complex *const y[], rocblas_int incy, rocblas_float_complex *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_zgeru_batched(rocblas_handle handle, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const x[], rocblas_int incx, const rocblas_double_complex *const y[], rocblas_int incy, rocblas_double_complex *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_cgerc_batched(rocblas_handle handle, rocblas_int m, rocblas_i

nt n, const rocblas_float_complex *alpha, const rocblas_float_complex *const x[], roc blas_int incx, const rocblas_float_complex *const y[], rocblas_int incy, rocblas_float_ complex *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_zgerc_batched(rocblas_handle handle, rocblas_int m, rocblas_i nt n, const rocblas_double_complex *alpha, const rocblas_double_complex *const x[] , rocblas_int incx, const rocblas_double_complex *const y[], rocblas_int incy, rocblas _double_complex *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_sger_strided_batched(rocblas_handle handle, rocblas_int m, ro cblas_int n, const float *alpha, const float *x, rocblas_int incx, rocblas_stride stridex, const float *y, rocblas_int incy, rocblas_stride stridey, float *A, rocblas_int lda, rocbla s_stride strideA, rocblas_int batch_count)

rocblas_status rocblas_dger_strided_batched(rocblas_handle handle, rocblas_int m, ro cblas_int n, const double *alpha, const double *x, rocblas_int incx, rocblas_stride stri dex, const double *y, rocblas_int incy, rocblas_stride stridey, double *A, rocblas_int l da, rocblas_stride strideA, rocblas_int batch_count)

rocblas_status rocblas_cgeru_strided_batched(rocblas_handle handle, rocblas_int m, r ocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x, ro cblas_int incx, rocblas_stride stridex, const rocblas_float_complex *y, rocblas_int inc y, rocblas_stride stridey, rocblas_float_complex *A, rocblas_int lda, rocblas_stride str ideA, rocblas_int batch_count)

rocblas_status rocblas_zgeru_strided_batched(rocblas_handle handle, rocblas_int m, r ocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex * x, rocblas_int incx, rocblas_stride stridex, const rocblas_double_complex *y, rocblas_ int incy, rocblas_stride stridey, rocblas_double_complex *A, rocblas_int lda, rocblas_ stride strideA, rocblas_int batch_count)

rocblas_status rocblas_cgerc_strided_batched(rocblas_handle handle, rocblas_int m, r ocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x, ro cblas_int incx, rocblas_stride stridex, const rocblas_float_complex *y, rocblas_int inc y, rocblas_stride stridey, rocblas_float_complex *A, rocblas_int lda, rocblas_stride str

ideA, rocblas_int batch_count)

rocblas_status rocblas_zgerc_strided_batched(rocblas_handle handle, rocblas_int m, r ocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex * x, rocblas_int incx, rocblas_stride stridex, const rocblas_double_complex *y, rocblas_ int incy, rocblas_stride stridey, rocblas_double_complex *A, rocblas_int lda, rocblas_ stride strideA, rocblas_int batch_count)

5.5.4 sbmv 函数族

sbmv 执行对称带状矩阵和向量之间的乘法操作。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
3) n：矩阵 A 的行数和列数；
4) k：指定子对角线和上对角线的数量；
5) alpha：指向标量 alpha 的设备指针或主机指针；
6) A：设备指针，存储矩阵 A；
7) lda：指定 A 的主维度；
8) x：存储向量 x 的设备指针；
9) incx：指定 x 元素的增量；
10) beta：指向标量 beta 的设备指针或主机指针；
11) y：存储向量 y 的设备指针；
12) incy：指定 y 元素的增量；
13) batch_count：批次数；
14) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
15) stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；
16) stridey：一个向量（y_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_ssbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const float *alpha, const float *A, rocblas_int lda, const float *x, rocbla s_int incx, const float *beta, float *y, rocblas_int incy)

rocblas_status rocblas_dsbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n,

rocblas_int k, const double *alpha, const double *A, rocblas_int lda, const double *x, rocblas_int incx, const double *beta, double *y, rocblas_int incy)

rocblas_status rocblas_ssbmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const float *alpha, const float *const A[], rocblas_int lda, const float *const x[], rocblas_int incx, const float *beta, float *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_dsbmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const double *alpha, const double *const A[], rocblas_int lda, const double *const x[], rocblas_int incx, const double *beta, double *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_ssbmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const float *alpha, const float *A, rocblas_int lda, rocblas_stride strideA, const float *x, rocblas_int incx, rocblas_stride stridex, const float *beta, float *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_dsbmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const double *alpha, const double *A, rocblas_int lda, rocblas_stride strideA, const double *x, rocblas_int incx, rocblas_stride stridex, const double *beta, double *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

5.5.5 spmv 函数族

spmv 执行对称压缩矩阵和向量之间的乘法操作。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
3) n：矩阵 A 的行数和列数；
4) k：指定子对角线和上对角线的数量；
5) alpha：指向标量 alpha 的设备指针或主机指针；
6) A：设备指针，存储矩阵 A；
7) lda：指定 A 的主维度；

8) x：存储向量 x 的设备指针；

9) incx：指定 x 元素的增量；

10) beta：指向标量 beta 的设备指针或主机指针；

11) y：存储向量 y 的设备指针；

12) incy：指定 y 元素的增量；

13) batch_count：批次数；

14) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；

15) stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

16) stridey：一个向量（y_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_sspmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const float *A, const float *x, rocblas_int incx, const float *beta, float *y, rocblas_int incy)

rocblas_status rocblas_dspmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const double *A, const double *x, rocblas_int incx, const double *beta, double *y, rocblas_int incy)

rocblas_status rocblas_sspmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const float *const A[], const float *const x[], rocblas_int incx, const float *beta, float *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_dspmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const double *const A[], const double *const x[], rocblas_int incx, const double *beta, double *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_sspmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const float *A, rocblas_stride strideA, const float *x, rocblas_int incx, rocblas_stride stridex, const float *beta, float *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_dspmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const double *A, rocblas_stride strideA, const double *x, rocblas_int incx, rocblas_stride stridex, const double *beta, double *y, roc

blas_int incy, rocblas_stride stridey, rocblas_int batch_count)

5.5.6 spr 函数族

spr 执行对称压缩矩阵的 rank-1 更新。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
3) n：矩阵 A 的行数和列数；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) x：存储向量 x 的设备指针；
6) incx：指定 x 元素的增量；
7) AP：设备指针，存储对称矩阵 A 的指定三角部分的压缩版本；
8) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_sspr(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, co
nst float *alpha, const float *x, rocblas_int incx, float *AP)

rocblas_status rocblas_dspr(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, co
nst double *alpha, const double *x, rocblas_int incx, double *AP)

rocblas_status rocblas_cspr(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, co
nst rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, r
ocblas_float_complex *AP)

rocblas_status rocblas_zspr(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, co
nst rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int in
cx, rocblas_double_complex *AP)

rocblas_status rocblas_sspr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas
_int n, const float *alpha, const float *const x[], rocblas_int incx, float *const AP[], ro
cblas_int batch_count)

rocblas_status rocblas_dspr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas
_int n, const double *alpha, const double *const x[], rocblas_int incx, double *const
AP[], rocblas_int batch_count)

rocblas_status rocblas_cspr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas

_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const x[], r
ocblas_int incx, rocblas_float_complex *const AP[], rocblas_int batch_count)
rocblas_status rocblas_zspr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas
_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const
x[], rocblas_int incx, rocblas_double_complex *const AP[], rocblas_int batch_count)
rocblas_status rocblas_sspr_strided_batched(rocblas_handle handle, rocblas_fill uplo,
rocblas_int n, const float *alpha, const float *x, rocblas_int incx, rocblas_stride strid
e_x, float *AP, rocblas_stride stride_A, rocblas_int batch_count)
rocblas_status rocblas_dspr_strided_batched(rocblas_handle handle, rocblas_fill uplo,
rocblas_int n, const double *alpha, const double *x, rocblas_int incx, rocblas_stride
stride_x, double *AP, rocblas_stride stride_A, rocblas_int batch_count)
rocblas_status rocblas_cspr_strided_batched(rocblas_handle handle, rocblas_fill uplo,
rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x,
rocblas_int incx, rocblas_stride stride_x, rocblas_float_complex *AP, rocblas_stride st
ride_A, rocblas_int batch_count)
rocblas_status rocblas_zspr_strided_batched(rocblas_handle handle, rocblas_fill uplo,
rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex
*x, rocblas_int incx, rocblas_stride stride_x, rocblas_double_complex *AP, rocblas_s
tride stride_A, rocblas_int batch_count)

5.5.7 spr2 函数族

spr2 执行对称压缩矩阵的 rank-2 更新。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
3) n：矩阵 A 的行数和列数；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) x：存储向量 x 的设备指针；
6) incx：指定 x 元素的增量；

7) AP：设备指针，存储对称矩阵 A 的指定三角部分的压缩版本；

8) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_sspr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const float *x, rocblas_int incx, const float *y, rocblas_int incy, float *AP)

rocblas_status rocblas_dspr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const double *x, rocblas_int incx, const double *y, rocblas_int incy, double *AP)

rocblas_status rocblas_sspr2_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const float *const x[], rocblas_int incx, const float *const y[], rocblas_int incy, float *const AP[], rocblas_int batch_count)

rocblas_status rocblas_dspr2_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const double *const x[], rocblas_int incx, const double *const y[], rocblas_int incy, double *const AP[], rocblas_int batch_count)

rocblas_status rocblas_sspr2_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const float *x, rocblas_int incx, rocblas_stride stride_x, const float *y, rocblas_int incy, rocblas_stride stride_y, float *AP, rocblas_stride stride_A, rocblas_int batch_count)

rocblas_status rocblas_dspr2_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const double *x, rocblas_int incx, rocblas_stride stride_x, const double *y, rocblas_int incy, rocblas_stride stride_y, double *AP, rocblas_stride stride_A, rocblas_int batch_count)

5.5.8 symv 函数族

`symv` 执行对称矩阵和向量乘法。

参数：

1) handle：上下文队列的句柄；

2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；

3) n：矩阵 A 的行数和列数；

4) alpha：指向标量 alpha 的设备指针或主机指针；

5) A：设备指针，在 GPU 上存储矩阵 A；

6) lda：指定 A 的主维度；

7) x：存储向量 x 的设备指针；

8) incx：指定 x 元素的增量；

9) beta：指向标量 beta 的设备指针或主机指针；

10) y：存储向量 y 的设备指针；

11) incy：指定 y 元素的增量；

12) batch_count：批次数；

13) strideA：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；

14) stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

15) stridey：一个向量（y_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_ssymv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, const float *x, rocblas_int incx, con st float *beta, float *y, rocblas_int incy)

rocblas_status rocblas_dsymv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, const double *x, rocblas_int in cx, const double *beta, double *y, rocblas_int incy)

rocblas_status rocblas_csymv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *bet a, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zsymv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_in t lda, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_comp lex *beta, rocblas_double_complex *y, rocblas_int incy)

rocblas_status rocblas_ssymv_batched(rocblas_handle handle, rocblas_fill uplo, rocbl as_int n, const float *alpha, const float *const A[], rocblas_int lda, const float *const x[], rocblas_int incx, const float *beta, float *const y[], rocblas_int incy, rocblas_int b

atch_count)

rocblas_status rocblas_dsymv_batched(rocblas_handle handle, rocblas_fill uplo, rocbl as_int n, const double *alpha, const double *const A[], rocblas_int lda, const double * const x[], rocblas_int incx, const double *beta, double *const y[], rocblas_int incy, roc blas_int batch_count)

rocblas_status rocblas_csymv_batched(rocblas_handle handle, rocblas_fill uplo, rocbl as_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[] , rocblas_int lda, const rocblas_float_complex *const x[], rocblas_int incx, const rocbl as_float_complex *beta, rocblas_float_complex *const y[], rocblas_int incy, rocblas_i nt batch_count)

rocblas_status rocblas_zsymv_batched(rocblas_handle handle, rocblas_fill uplo, rocbl as_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *cons t A[], rocblas_int lda, const rocblas_double_complex *const x[], rocblas_int incx, con st rocblas_double_complex *beta, rocblas_double_complex *const y[], rocblas_int in cy, rocblas_int batch_count)

rocblas_status rocblas_ssymv_strided_batched(rocblas_handle handle, rocblas_fill upl o, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, rocblas_stride strid eA, const float *x, rocblas_int incx, rocblas_stride stridex, const float *beta, float *y, r ocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_dsymv_strided_batched(rocblas_handle handle, rocblas_fill up lo, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, rocblas_stride strideA, const double *x, rocblas_int incx, rocblas_stride stridex, const double *beta, double *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_csymv_strided_batched(rocblas_handle handle, rocblas_fill up lo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex * A, rocblas_int lda, rocblas_stride strideA, const rocblas_float_complex *x, rocblas_in t incx, rocblas_stride stridex, const rocblas_float_complex *beta, rocblas_float_compl ex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_zsymv_strided_batched(rocblas_handle handle, rocblas_fill up

lo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_compl ex *A, rocblas_int lda, rocblas_stride strideA, const rocblas_double_complex *x, rocb las_int incx, rocblas_stride stridex, const rocblas_double_complex *beta, rocblas_dou ble_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

5.5.9 syr 函数族

syr 执行对称矩阵的 rank-1 更新。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
3) n：矩阵 A 的行数和列数；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) x：存储向量 x 的设备指针；
6) incx：指定 x 元素的增量；
7) A：设备指针，存储对称矩阵 A ；
8) lda：指定 A 的主维度；
9) batch_count：批次数；
10) strideA：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；

rocblas_status rocblas_zsymv_strided_batched(rocblas_handle handle, rocblas_fill up lo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_compl ex *A, rocblas_int lda, rocblas_stride strideA, const rocblas_double_complex *x, rocb las_int incx, rocblas_stride stridex, const rocblas_double_complex *beta, rocblas_dou ble_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count)

rocblas_status rocblas_dsyr(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, co nst double *alpha, const double *x, rocblas_int incx, double *A, rocblas_int lda)

rocblas_status rocblas_csyr(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, co nst rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, r ocblas_float_complex *A, rocblas_int lda)

rocblas_status rocblas_zsyr(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, co

nst rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int in cx, rocblas_double_complex *A, rocblas_int lda)

rocblas_status rocblas_ssyr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas _int n, const float *alpha, const float *const x[], rocblas_int incx, float *const A[], roc blas_int lda, rocblas_int batch_count)

rocblas_status rocblas_dsyr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas _int n, const double *alpha, const double *const x[], rocblas_int incx, double *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_csyr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas _int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const x[], r ocblas_int incx, rocblas_float_complex *const A[], rocblas_int lda, rocblas_int batch_ count)

rocblas_status rocblas_zsyr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas _int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const x[], rocblas_int incx, rocblas_double_complex *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_ssyr_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const float *x, rocblas_int incx, rocblas_stride strid ex, float *A, rocblas_int lda, rocblas_stride strideA, rocblas_int batch_count)

rocblas_status rocblas_dsyr_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const double *x, rocblas_int incx, rocblas_stride stridex, double *A, rocblas_int lda, rocblas_stride strideA, rocblas_int batch_count)

rocblas_status rocblas_csyr_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_float_complex *A, rocblas_int lda, ro cblas_stride strideA, rocblas_int batch_count)

rocblas_status rocblas_zsyr_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int incx, rocblas_stride stridex, rocblas_double_complex *A, rocblas_int

lda, rocblas_stride strideA, rocblas_int batch_count)

5.5.10 syr2 函数族

syr2 执行对称矩阵的 rank-2 更新。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
3) n：矩阵 A 的行数和列数；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) x：存储向量 x 的设备指针；
6) incx：指定 x 元素的增量；
7) A：设备指针，存储对称矩阵 A ；
8) lda：指定 A 的主维度；
9) batch_count：批次数；
10) strideA：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；

rocblas_status rocblas_ssyr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const float *x, rocblas_int incx, const float *y, rocblas_int incy, float *A, rocblas_int lda)

rocblas_status rocblas_dsyr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const double *x, rocblas_int incx, const double *y, rocblas_int incy, double *A, rocblas_int lda)

rocblas_status rocblas_csyr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *y, rocblas_int incy, rocblas_float_complex *A, rocblas_int lda)

rocblas_status rocblas_zsyr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *y, rocblas_int incy, rocblas_double_complex *A, rocblas_int lda)

rocblas_status rocblas_ssyr2_batched(rocblas_handle handle, rocblas_fill uplo, rocbla
s_int n, const float *alpha, const float *const x[], rocblas_int incx, const float *const y
[], rocblas_int incy, float *const A[], rocblas_int lda, rocblas_int batch_count)
rocblas_status rocblas_dsyr2_batched(rocblas_handle handle, rocblas_fill uplo, rocbla
s_int n, const double *alpha, const double *const x[], rocblas_int incx, const double *
const y[], rocblas_int incy, double *const A[], rocblas_int lda, rocblas_int batch_count)
rocblas_status rocblas_csyr2_batched(rocblas_handle handle, rocblas_fill uplo, rocbla
s_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const x[],
rocblas_int incx, const rocblas_float_complex *const y[], rocblas_int incy, rocblas_flo
at_complex *const A[], rocblas_int lda, rocblas_int batch_count)
rocblas_status rocblas_zsyr2_batched(rocblas_handle handle, rocblas_fill uplo, rocbla
s_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const
x[], rocblas_int incx, const rocblas_double_complex *const y[], rocblas_int incy, rocb
las_double_complex *const A[], rocblas_int lda, rocblas_int batch_count)
rocblas_status rocblas_ssyr2_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_int n, const float *alpha, const float *x, rocblas_int incx, rocblas_stride stri
dex, const float *y, rocblas_int incy, rocblas_stride stridey, float *A, rocblas_int lda, r
ocblas_stride strideA, rocblas_int batch_count)
rocblas_status rocblas_dsyr2_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_int n, const double *alpha, const double *x, rocblas_int incx, rocblas_stride
stridex, const double *y, rocblas_int incy, rocblas_stride stridey, double *A, rocblas_i
nt lda, rocblas_stride strideA, rocblas_int batch_count)
rocblas_status rocblas_csyr2_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x
, rocblas_int incx, rocblas_stride stridex, const rocblas_float_complex *y, rocblas_int
incy, rocblas_stride stridey, rocblas_float_complex *A, rocblas_int lda, rocblas_stride
strideA, rocblas_int batch_count)
rocblas_status rocblas_zsyr2_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_comple

x *x, rocblas_int incx, rocblas_stride stridex, const rocblas_double_complex *y, rocblas_int incy, rocblas_stride stridey, rocblas_double_complex *A, rocblas_int lda, rocblas_stride strideA, rocblas_int batch_count)

5.5.11 tbmv 函数族

tbmv 计算带状三角矩阵与向量的乘积。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) trans：表示矩阵 A 是否被转置（共轭）；
4) diag：指定 A 是否为单位三角矩阵，“rocblas_diagonal_unit”A 的主对角线被认为仅由 1 组成，并且未被引用，“rocblas_diagonal_non_unit”没有对 A 的主对角线做出任何假设；
5) m：矩阵 A 的行数和列数；
6) k：带状矩阵的带宽范围；
7) A：设备指针，带状三角矩阵 A；
8) lda：指定 A 的主维度；
9) x：存储向量 x 的设备指针；
10) incx：指定 x 元素的增量；
11) batch_count：批次数；
12) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
13) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_stbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int m, rocblas_int k, const float *A, rocblas_int lda, float *x, rocblas_int incx)

rocblas_status rocblas_dtbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int m, rocblas_int k, const double *A, rocblas_int lda, double *x, rocblas_int incx)

rocblas_status rocblas_ctbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int m, rocblas_int k, const rocblas_float_co

mplex *A, rocblas_int lda, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_ztbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int m, rocblas_int k, const rocblas_double_complex *A, rocblas_int lda, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_stbmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const float *const A[], rocblas_int lda, float *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_dtbmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const double *const A[], rocblas_int lda, double *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ctbmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const rocblas_float_complex *const A[], rocblas_int lda, rocblas_float_complex *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ztbmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const rocblas_double_complex *const A[], rocblas_int lda, rocblas_double_complex *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_stbmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const float *A, rocblas_int lda, rocblas_stride stride_A, float *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_dtbmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const double *A, rocblas_int lda, rocblas_stride stride_A, double *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_ctbmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const r

ocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_float_com
plex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)
rocblas_status rocblas_ztbmv_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_operation trans, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const r
ocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_double_
complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

5.5.12 tbsv 函数族

tbsv 求解带状三角矩阵 A。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) transA：表示矩阵 A 是否被转置（共轭）；
4) diag：指定 A 是否为单位三角矩阵，“rocblas_diagonal_unit”A 的主对角线被认为仅由 1 组成，并且未被引用，“rocblas_diagonal_non_unit”没有对 A 的主对角线做出任何假设；
5) n：n 指定 b 的行数；
6) k：带状矩阵的带宽范围；
7) A：设备指针，带状三角矩阵 A；
8) lda：指定 A 的主维度；
9) x：存储向量 x 的设备指针；
10) incx：指定 x 元素的增量；
11) batch_count：批次数；
12) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
13) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_stbsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operati
on transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const float *A, rocblas
_int lda, float *x, rocblas_int incx)
rocblas_status rocblas_dtbsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operati
on transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const double *A, rocbl

as_int lda, double *x, rocblas_int incx)

rocblas_status rocblas_ctbsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const rocblas_float_complex *A, rocblas_int lda, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_ztbsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const rocblas_double_complex *A, rocblas_int lda, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_stbsv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const float *const A[], rocblas_int lda, float *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_dtbsv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const double *const A[], rocblas_int lda, double *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ctbsv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const rocblas_float_complex *const A[], rocblas_int lda, rocblas_float_complex *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ztbsv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const rocblas_double_complex *const A[], rocblas_int lda, rocblas_double_complex *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_stbsv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const float *A, rocblas_int lda, rocblas_stride stride_A, float *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_dtbsv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const double *A, rocblas_int lda, rocblas_stride stride_A, double *x, rocblas_int incx, rocb

las_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_ctbsv_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const
rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_float_co
mplex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_ztbsv_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const
rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_doubl
e_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

5.5.13 tpmv 函数族

tbmv 计算压缩三角矩阵与向量的乘积。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) transA：表示矩阵 A 是否被转置（共轭）；
4) diag：指定 A 是否为单位三角矩阵，“rocblas_diagonal_unit”A 的主对角线被认为仅由 1 组成，并且未被引用，“rocblas_diagonal_non_unit”没有对 A 的主对角线做出任何假设；
5) n：n 指定 A 的行数；
6) A：设备指针，存储压缩三角矩阵 A；
7) x：存储向量 x 的设备指针；
8) incx：指定 x 元素的增量；
9) batch_count：批次数；
10) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
11) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_ztbsv_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, rocblas_int k, const
rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_doubl
e_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_stpmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const float *A, float *x, rocblas_int incx)

rocblas_status rocblas_dtpmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const double *A, double *x, rocblas_int incx)

rocblas_status rocblas_ctpmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *A, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_ztpmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_complex *A, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_dtpmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const double *const *A, double *const *x, rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ctpmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *const *A, rocblas_float_complex *const *x, rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ztpmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_complex *const *A, rocblas_double_complex *const *x, rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ztpmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_complex *const *A, rocblas_double_complex *const *x, rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_stpmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const float *A, rocb

las_stride stride_A, float *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_stpmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const float *A, rocblas_stride stride_A, float *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_dtpmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const double *A, rocblas_stride stride_A, double *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_ctpmv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *A, rocblas_stride stride_A, rocblas_float_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

5.5.14 tpsv 函数族

tbsv 求解压缩三角矩阵 A。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) transA：表示矩阵 A 是否被转置（共轭）；
4) diag：指定 A 是否为单位三角矩阵，“rocblas_diagonal_unit”A 的主对角线被认为仅由 1 组成，并且未被引用，“rocblas_diagonal_non_unit”没有对 A 的主对角线做出任何假设；
5) n：n 指定 A 的行数；
6) AP：设备指针，存储三角矩阵 A 的指定三角部分的压缩版本；
7) x：存储向量 x 的设备指针；
8) incx：指定 x 元素的增量；
9) batch_count：批次数；
10) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；

11）stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_stpsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const float *AP, float *x, rocblas_int incx)

rocblas_status rocblas_dtpsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const double *AP, double *x, rocblas_int incx)

rocblas_status rocblas_ctpsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *AP, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_ztpsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_complex *AP, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_stpsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const float *AP, float *x, rocblas_int incx)

rocblas_status rocblas_dtpsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const double *AP, double *x, rocblas_int incx)

rocblas_status rocblas_ctpsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *AP, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_ztpsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_complex *AP, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_stpsv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const float *const AP[], float *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_dtpsv_batched(rocblas_handle handle, rocblas_fill uplo, rocbla

s_operation transA, rocblas_diagonal diag, rocblas_int n, const double *const AP[], d ouble *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ctpsv_batched(rocblas_handle handle, rocblas_fill uplo, rocbla s_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *const AP[], rocblas_float_complex *const x[], rocblas_int incx, rocblas_int batch_c ount)

rocblas_status rocblas_ztpsv_batched(rocblas_handle handle, rocblas_fill uplo, rocbla s_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_compl ex *const AP[], rocblas_double_complex *const x[], rocblas_int incx, rocblas_int batc h_count)

5.5.15 trmv 函数族

trmv 计算三角矩阵与向量的乘积。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) transA：表示矩阵 A 是否被转置（共轭）；
4) diag：指定 A 是否为单位三角矩阵，“rocblas_diagonal_unit”A 的主对角线被认为仅由 1 组成，并且未被引用，“rocblas_diagonal_non_unit”没有对 A 的主对角线做出任何假设；
5) n：n 指定 A 的行数；
6) A：设备指针，存储矩阵 A；
7) lda：指定 A 的主维度；
8) x：存储向量 x 的设备指针；
9) incx：指定 x 元素的增量；
10) batch_count：批次数；
11) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
12) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_strmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operat ion transA, rocblas_diagonal diag, rocblas_int n, const float *A, rocblas_int lda, float

*x, rocblas_int incx)

rocblas_status rocblas_dtrmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operat ion transA, rocblas_diagonal diag, rocblas_int n, const double *A, rocblas_int lda, do uble *x, rocblas_int incx)

rocblas_status rocblas_ctrmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operat ion transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *A, roc blas_int lda, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_ztrmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operat ion transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_complex *A, r ocblas_int lda, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_strmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operat ion transA, rocblas_diagonal diag, rocblas_int n, const float *A, rocblas_int lda, float *x, rocblas_int incx)

rocblas_status rocblas_dtrmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operat ion transA, rocblas_diagonal diag, rocblas_int n, const double *A, rocblas_int lda, do uble *x, rocblas_int incx)

rocblas_status rocblas_ctrmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operat ion transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *A, roc blas_int lda, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_ztrmv(rocblas_handle handle, rocblas_fill uplo, rocblas_operat ion transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_complex *A, r ocblas_int lda, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_strmv_batched(rocblas_handle handle, rocblas_fill uplo, rocbla s_operation transA, rocblas_diagonal diag, rocblas_int n, const float *const *A, rocbla s_int lda, float *const *x, rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_dtrmv_batched(rocblas_handle handle, rocblas_fill uplo, rocbl as_operation transA, rocblas_diagonal diag, rocblas_int n, const double *const *A, ro cblas_int lda, double *const *x, rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ctrmv_batched(rocblas_handle handle, rocblas_fill uplo, rocbl

as_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_comple x *const *A, rocblas_int lda, rocblas_float_complex *const *x, rocblas_int incx, rocbl as_int batch_count)

rocblas_status rocblas_ztrmv_batched(rocblas_handle handle, rocblas_fill uplo, rocbl as_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_com plex *const *A, rocblas_int lda, rocblas_double_complex *const *x, rocblas_int incx, rocblas_int batch_count)

5.5.16 trsv 函数族

trsv 求解三角矩阵：

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) transA：表示矩阵 A 是否被转置（共轭）；
4) diag：指定 A 是否为单位三角矩阵，“rocblas_diagonal_unit”A 的主对角线被认为仅由 1 组成，并且未被引用，“rocblas_diagonal_non_unit”没有对 A 的主对角线做出任何假设；
5) n：n 指定 b 的行数；
6) A：设备指针，存储矩阵 A；
7) lda：指定 A 的主维度；
8) x：存储向量 x 的设备指针；
9) incx：指定 x 元素的增量；
10) batch_count：批次数；
11) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
12) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_strmv_strided_batched(rocblas_handle handle, rocblas_fill upl o, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const float *A, rocb las_int lda, rocblas_stride stride_A, float *x, rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_dtrmv_strided_batched(rocblas_handle handle, rocblas_fill upl

o, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const double *A, ro
cblas_int lda, rocblas_stride stride_A, double *x, rocblas_int incx, rocblas_stride strid
e_x, rocblas_int batch_count)

rocblas_status rocblas_ctrmv_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float
_complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_float_complex *x, roc
blas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_ztrmv_strided_batched(rocblas_handle handle, rocblas_fill upl
o, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_doubl
e_complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_double_complex *x,
rocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_strsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operati
on transA, rocblas_diagonal diag, rocblas_int n, const float *A, rocblas_int lda, float *
x, rocblas_int incx)

rocblas_status rocblas_dtrsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operati
on transA, rocblas_diagonal diag, rocblas_int n, const double *A, rocblas_int lda, dou
ble *x, rocblas_int incx)

rocblas_status rocblas_ctrsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operati
on transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *A, roc
blas_int lda, rocblas_float_complex *x, rocblas_int incx)

rocblas_status rocblas_ztrsv(rocblas_handle handle, rocblas_fill uplo, rocblas_operati
on transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_complex *A, r
ocblas_int lda, rocblas_double_complex *x, rocblas_int incx)

rocblas_status rocblas_strsv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas
_operation transA, rocblas_diagonal diag, rocblas_int n, const float *const A[], rocbla
s_int lda, float *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_dtrsv_batched(rocblas_handle handle, rocblas_fill uplo, rocbla
s_operation transA, rocblas_diagonal diag, rocblas_int n, const double *const A[], roc
blas_int lda, double *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ctrsv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_complex *const A[], rocblas_int lda, rocblas_float_complex *const x[], rocblas_int incx, rocblas_int batch_count)

rocblas_status rocblas_ztrsv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double_complex *const A[], rocblas_int lda, rocblas_double_complex *const x[], rocblas_int incx, rocblas_int batch_count)

5.5.17 hemv 函数族

hemv 执行 Hermitian 矩阵与向量的乘积。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) n：矩阵 A 的阶；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) A：存储矩阵 A 的设备指针；
6) lda：指定 A 的主维度；
7) x：存储向量 x 的设备指针；
8) incx：指定 x 元素的增量；
9) beta：指向标量 beta 的设备指针或主机指针；
10) y：存储向量 y 的设备指针；
11) incy：指定 y 元素的增量；
12) batch_count：批次数；
13) strideA：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
14) stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；
15) stridey：一个向量（y_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_strsv_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const float *A, rocblas_int lda, rocblas_stride stride_A, float *x, rocblas_int incx, rocblas_stride stride_x, r

开发者社区

ocblas_int batch_count)

rocblas_status rocblas_dtrsv_strided_batched(rocblas_handle handle, rocblas_fill uplo , rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const double *A, roc blas_int lda, rocblas_stride stride_A, double *x, rocblas_int incx, rocblas_stride stride _x, rocblas_int batch_count)

rocblas_status rocblas_ctrsv_strided_batched(rocblas_handle handle, rocblas_fill uplo , rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_float_c omplex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_float_complex *x, rocbla s_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_ztrsv_strided_batched(rocblas_handle handle, rocblas_fill uplo , rocblas_operation transA, rocblas_diagonal diag, rocblas_int n, const rocblas_double _complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_double_complex *x, r ocblas_int incx, rocblas_stride stride_x, rocblas_int batch_count)

rocblas_status rocblas_chemv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *bet a, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zhemv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_in t lda, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_comp lex *beta, rocblas_double_complex *y, rocblas_int incy)

rocblas_status rocblas_chemv_strided_batched(rocblas_handle handle, rocblas_fill up lo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex * A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *x, rocblas_i nt incx, rocblas_stride stride_x, const rocblas_float_complex *beta, rocblas_float_co mplex *y, rocblas_int incy, rocblas_stride stride_y, rocblas_int batch_count)

rocblas_status rocblas_zhemv_strided_batched(rocblas_handle handle, rocblas_fill up lo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_compl ex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *x, roc

blas_int incx, rocblas_stride stride_x, const rocblas_double_complex *beta, rocblas_double_complex *y, rocblas_int incy, rocblas_stride stride_y, rocblas_int batch_count)

5.5.18 hbmv 函数族

hbmv 执行 Hermitian 带状矩阵与向量的乘积。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) n：矩阵 A 的阶；
4) k：矩阵 A 的上对角线的数量；
5) alpha：指向标量 alpha 的设备指针或主机指针；
6) A：存储矩阵 A 的设备指针；
7) lda：指定 A 的主维度；
8) x：存储向量 x 的设备指针；
9) incx：指定 x 元素的增量；
10) beta：指向标量 beta 的设备指针或主机指针；
11) y：存储向量 y 的设备指针；
12) incy：指定 y 元素的增量；
13) batch_count：批次数；
14) strideA：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
15) stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；
16) stridey：一个向量（y_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_chbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zhbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *y, rocblas_int incy)

开发者社区

rocblas_status rocblas_chbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zhbmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *y, rocblas_int incy)

rocblas_status rocblas_chbmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const x[], rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_zhbmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const x[], rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *const y[], rocblas_int incy, rocblas_int batch_count)

5.5.19 hpmv 函数族

hpmv 执行 Hermitian 压缩矩阵与向量的乘积。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) n：矩阵 A 的阶；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) AP：设备指针，存储 Hermitian 矩阵 A 的指定三角部分的压缩版本；
6) x：存储向量 x 的设备指针；
7) incx：指定 x 元素的增量；
8) beta：指向标量 beta 的设备指针或主机指针；

9) y：存储向量 y 的设备指针；

10) incy：指定 y 元素的增量；

11) batch_count：批次数；

12）strideA：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；

13）stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

14）stridey：一个向量（y_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_chpmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *AP, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zhpmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *AP, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *y, rocblas_int incy)

rocblas_status rocblas_chpmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *AP, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *y, rocblas_int incy)

rocblas_status rocblas_zhpmv(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *AP, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *y, rocblas_int incy)

rocblas_status rocblas_chpmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const AP[], const rocblas_float_complex *const x[], rocblas_int incx, const rocblas_float_complex *beta, rocblas_float_complex *const y[], rocblas_int incy, rocblas_int batch_count)

rocblas_status rocblas_zhpmv_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *cons

t AP[], const rocblas_double_complex *const x[], rocblas_int incx, const rocblas_double_complex *beta, rocblas_double_complex *const y[], rocblas_int incy, rocblas_int batch_count)

5.5.20 her 函数族

her 使用向量 x 对 Hermitian 矩阵 A 执行 rank-1 更新。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) n：矩阵 A 的行数和列数；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) x：存储向量 x 的设备指针；
6) incx：指定 x 元素的增量；
7) A：存储 Hermitian 矩阵 A 的指定三角部分；
8) lda：指定 A 的主维度；
9) batch_count：批次数；
10) strideA：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
11) stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_cher(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const rocblas_float_complex *x, rocblas_int incx, rocblas_float_complex *A, rocblas_int lda)

rocblas_status rocblas_zher(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const rocblas_double_complex *x, rocblas_int incx, rocblas_double_complex *A, rocblas_int lda)

rocblas_status rocblas_cher(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const rocblas_float_complex *x, rocblas_int incx, rocblas_float_complex *A, rocblas_int lda)

rocblas_status rocblas_zher(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const rocblas_double_complex *x, rocblas_int incx, rocblas_double_complex *A, rocblas_int lda)

开发者社区

rocblas_status rocblas_cher_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const rocblas_float_complex *const x[], rocblas_int incx, rocblas_float_complex *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_zher_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const rocblas_double_complex *const x[], rocblas_int incx, rocblas_double_complex *const A[], rocblas_int lda, rocblas_int batch_count)

5.5.21 her2 函数族

her2 使用向量 x 、y 对 Hermitian 矩阵 A 执行 rank-2 更新。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) n：矩阵 A 的行数和列数；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) x：存储向量 x 的设备指针；
6) incx：指定 x 元素的增量；
7) A：存储 Hermitian 矩阵 A 的指定三角部分；
8) lda：指定 A 的主维度；
9) batch_count：批次数；
10) strideA：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
11) stridex：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_cher_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const float *alpha, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_int batch_count)

rocblas_status rocblas_zher_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const double *alpha, const rocblas_double_complex *x, rocblas_int incx, rocblas_stride stride_x, rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, rocblas_int batch_count)

rocblas_status rocblas_cher2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, c

onst rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *y, rocblas_int incy, rocblas_float_complex *A, rocblas_int lda)

rocblas_status rocblas_zher2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *y, rocblas_int incy, rocblas_double_complex *A, rocblas_int lda)

rocblas_status rocblas_cher2_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const x[], rocblas_int incx, const rocblas_float_complex *const y[], rocblas_int incy, rocblas_float_complex *const A[], rocblas_int lda, rocblas_int batch_count)

rocblas_status rocblas_zher2_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const x[], rocblas_int incx, const rocblas_double_complex *const y[], rocblas_int incy, rocblas_double_complex *const A[], rocblas_int lda, rocblas_int batch_count)

5.5.22 hpr 函数族

hpr 对 Hermitian 矩阵 以压缩形式进行 rank-1 更新。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) n：矩阵 A 的行数和列数；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) AP：设备指针，存储 Hermitian 矩阵 A 的指定三角部分的压缩版本；
6) x：存储向量 x 的设备指针；
7) incx：指定 x 元素的增量；
8) batch_count：批次数；
9) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；
10) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_cher2_strided_batched(rocblas_handle handle, rocblas_fill upl

o, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x , rocblas_int incx, rocblas_stride stride_x, const rocblas_float_complex *y, rocblas_in t incy, rocblas_stride stride_y, rocblas_float_complex *A, rocblas_int lda, rocblas_stri de stride_A, rocblas_int batch_count)

rocblas_status rocblas_zher2_strided_batched(rocblas_handle handle, rocblas_fill upl o, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_comple x *x, rocblas_int incx, rocblas_stride stride_x, const rocblas_double_complex *y, roc blas_int incy, rocblas_stride stride_y, rocblas_double_complex *A, rocblas_int lda, ro cblas_stride stride_A, rocblas_int batch_count)

rocblas_status rocblas_chpr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas _int n, const float *alpha, const rocblas_float_complex *const x[], rocblas_int incx, ro cblas_float_complex *const AP[], rocblas_int batch_count)

rocblas_status rocblas_zhpr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas _int n, const double *alpha, const rocblas_double_complex *const x[], rocblas_int inc x, rocblas_double_complex *const AP[], rocblas_int batch_count)

rocblas_status rocblas_chpr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas _int n, const float *alpha, const rocblas_float_complex *const x[], rocblas_int incx, ro cblas_float_complex *const AP[], rocblas_int batch_count)

rocblas_status rocblas_zhpr_batched(rocblas_handle handle, rocblas_fill uplo, rocblas _int n, const double *alpha, const rocblas_double_complex *const x[], rocblas_int inc x, rocblas_double_complex *const AP[], rocblas_int batch_count)

5.5.23 hpr2 函数族

hpr2 对 Hermitian 矩阵 以压缩形式进行 rank-2 更新。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) n：矩阵 A 的行数和列数；
4) alpha：指向标量 alpha 的设备指针或主机指针；
5) AP：设备指针，存储 Hermitian 矩阵 A 的指定三角部分的压缩版本；

6) x：存储向量 x 的设备指针；

7) incx：指定 x 元素的增量；

8) batch_count：批次数；

9) stride_A：一个矩阵（A_i）的开始到下一个矩阵（A_i+1）的步幅；

10) stride_x：一个向量（x_i）的起点到下一个向量（x_i+1）的步幅；

rocblas_status rocblas_chpr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *y, rocblas_int incy, rocblas_float_complex *AP)

rocblas_status rocblas_zhpr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *y, rocblas_int incy, rocblas_double_complex *AP)

rocblas_status rocblas_chpr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *x, rocblas_int incx, const rocblas_float_complex *y, rocblas_int incy, rocblas_float_complex *AP)

rocblas_status rocblas_zhpr2(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *x, rocblas_int incx, const rocblas_double_complex *y, rocblas_int incy, rocblas_double_complex *AP)

rocblas_status rocblas_chpr2_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const x[], rocblas_int incx, const rocblas_float_complex *const y[], rocblas_int incy, rocblas_float_complex *const AP[], rocblas_int batch_count)

rocblas_status rocblas_zhpr2_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const x[], rocblas_int incx, const rocblas_double_complex *const y[], rocblas_int incy, rocblas_double_complex *const AP[], rocblas_int batch_count)

5.6 rocBLAS 三级函数

5.6.1 gemm 函数族

通用矩阵乘法 C = alpha*op(A)*op(B) + beta*C,，支持多种数据类型与批处理。

参数：

1) handle：上下文队列的句柄；
2) transA：对矩阵 A 是否进行转置；
3) transB：对矩阵 B 是否进行转置；
4) m：矩阵维度 m；
5) n：矩阵维度 n；
6) k：矩阵维度 k；
7) alpha：指向标量 alpha 的设备指针或主机指针；
8) A：A 矩阵指针, 矩阵 A 的维度是 m × k；
9) lda：指定 A 的主维度；
10) B：B 矩阵指针, 矩阵 B 的维度是 k × n；
11) ldb：指定 B 的主维度；
12) beta：指向标量 beta 的设备指针或主机指针；
13) C：C 矩阵指针,矩阵 C 的维度是 m × n；
14) ldc：指定 C 的主维度；
15) batch_count：批次数；
16) strida_a：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；
17) strida_b：从一个 B_i 矩阵的开始到下一个 B_(i + 1) 的步幅；
18) strida_c：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_sgemm(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const float *alpha, const float *A, rocblas_int lda, const float *B, rocblas_int ldb, const float *beta, float *C, rocblas_int ldc)

rocblas_status rocblas_dgemm(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const double

*alpha, const double *A, rocblas_int lda, const double *B, rocblas_int ldb, const double *beta, double *C, rocblas_int ldc)

rocblas_status rocblas_hgemm(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const rocblas_half *alpha, const rocblas_half *A, rocblas_int lda, const rocblas_half *B, rocblas_int ldb, const rocblas_half *beta, rocblas_half *C, rocblas_int ldc)

rocblas_status rocblas_cgemm(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *B, rocblas_int ldb, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zgemm(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *B, rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_sgemm_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const float *alpha, const float *const A[], rocblas_int lda, const float *const B[], rocblas_int ldb, const float *beta, float *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_dgemm_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const double *alpha, const double *const A[], rocblas_int lda, const double *const B[], rocblas_int ldb, const double *beta, double *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_hgemm_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const rocblas_half *alpha, const rocblas_half *const A[], rocblas_int lda, const rocblas_half *const B[], rocblas_int ldb, const rocblas_half *beta, rocblas_half *const C[],

rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_cgemm_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const B[], rocblas_int ldb, const rocblas_float_complex *beta, rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zgemm_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const B[], rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_sgemm_strided_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const float *alpha, const float *A, rocblas_int lda, rocblas_stride stride_a, const float *B, rocblas_int ldb, rocblas_stride stride_b, const float *beta, float *C, rocblas_int ldc, rocblas_stride stride_c, rocblas_int batch_count)

rocblas_status rocblas_dgemm_strided_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const double *alpha, const double *A, rocblas_int lda, rocblas_stride stride_a, const double *B, rocblas_int ldb, rocblas_stride stride_b, const double *beta, double *C, rocblas_int ldc, rocblas_stride stride_c, rocblas_int batch_count)

rocblas_status rocblas_hgemm_strided_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const rocblas_half *alpha, const rocblas_half *A, rocblas_int lda, rocblas_stride stride_a, const rocblas_half *B, rocblas_int ldb, rocblas_stride stride_b, const rocblas_half *beta, rocblas_half *C, rocblas_int ldc, rocblas_stride stride_c, rocblas_int batch_count)

rocblas_status rocblas_cgemm_strided_batched(rocblas_handle handle,

rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_a, const rocblas_float_complex *B, rocblas_int ldb, rocblas_stride stride_b, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_c, rocblas_int batch_count)

rocblas_status rocblas_zgemm_strided_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_a, const rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_b, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_c, rocblas_int batch_count)

5.6.2 symm 函数族

对称矩阵与一般矩阵乘法：

C ：= alpha*A*B + beta*C if side == rocblas_side_left,

C ：= alpha*B*A + beta*C if side == rocblas_side_right,

适用于结构优化。

参数：

1) handle：上下文队列的句柄；
2) side：指定三角矩阵 A 在乘法中的位置“rocblas_side_left”和“rocblas_side_right”；
3) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
4) m：指定矩阵 B 和矩阵 C 的行数；
5) n：指定矩阵 B 和矩阵 C 的列数；
6) alpha：指向标量 alpha 的设备指针或主机指针；
7) A：存储三角矩阵 A 的指针；
8) lda：指定 A 的主维度，每一列在内存中的跨度；
9) B：存储的矩阵 B 的指针，矩阵 B 的维度是 m × n；
10) ldb：指定 B 的主维度，每一列在内存中的跨度；

11) beta：指向标量 beta 的设备指针或主机指针；

12) C：指向 GPU 上存储的矩阵 C 的指针，矩阵 C 的维度是 m × n，存放最终的计算结果；

13) ldc：指定 C 的主维度，每一列在内存中的跨度；

14) batch_count：批次数；

15) strida_A：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；

16) strida_B：从一个 B_i 矩阵的开始到下一个 B_(i + 1) 的步幅；

17) strida_C：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_ssymm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, const float *B, rocblas_int ldb, const float *beta, float *C, rocblas_int ldc)

rocblas_status rocblas_dsymm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, const double *B, rocblas_int ldb, const double *beta, double *C, rocblas_int ldc)

rocblas_status rocblas_csymm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *B, rocblas_int ldb, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zsymm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *B, rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_ssymm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const float *alpha, const float *const A[], rocblas_int lda, const float *const B[], rocblas_int ldb, const float *beta, float *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_dsymm_batched(rocblas_handle handle, rocblas_side side,

rocblas_fill uplo, rocblas_int m, rocblas_int n, const double *alpha, const double *const A[], rocblas_int lda, const double *const B[], rocblas_int ldb, const double *beta, double *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_csymm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const B[], rocblas_int ldb, const rocblas_float_complex *beta, rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zsymm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const B[], rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_ssymm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, rocblas_stride stride_A, const float *B, rocblas_int ldb, rocblas_stride stride_B, const float *beta, float *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_dsymm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, rocblas_stride stride_A, const double *B, rocblas_int ldb, rocblas_stride stride_B, const double *beta, double *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_csymm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *B, rocblas_int ldb, rocblas_stride stride_B, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_zsymm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_B, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.6.3 syrk 函数族

对称矩阵的秩更新 C ：=alpha*op(A)*op(A)^T+beta*C，适用于结构保持的乘法。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) transA：对矩阵 A 是否进行转置；
4) n：指定矩阵 C 的**行数和列数**，即 C 是 n × n 的对称矩阵；
5) k：指定矩阵 op(A) 的列数；
6) alpha：指向标量 alpha 的设备指针或主机指针；
7) A： 指向 GPU 上存储的矩阵 A 的指针；
8) lda：指定 A 的主维度，每一列在内存中的跨度；
9) beta：指向标量 beta 的设备指针或主机指针；
10) C： 指向 GPU 上存储的矩阵 C 的指针，是一个 n × n 的对称矩阵；
11) ldc： 指定 C 的主维度，每一列在内存中的跨度；
12) batch_count：批次数；
13) strida_A：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；
14) strida_C：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_ssyrk(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const float *alpha, const float *A, rocblas_int lda, const float *beta, float *C, rocblas_int ldc)

rocblas_status rocblas_dsyrk(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const double *alpha, const

double *A, rocblas_int lda, const double *beta, double *C, rocblas_int ldc)

rocblas_status rocblas_csyrk(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zsyrk(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_ssyrk_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const float *alpha, const float *const A[], rocblas_int lda, const float *beta, float *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_dsyrk_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const double *alpha, const double *const A[], rocblas_int lda, const double *beta, double *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_csyrk_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *beta, rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zsyrk_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *beta, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_ssyrk_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const float *alpha, const float

*A, rocblas_int lda, rocblas_stride stride_A, const float *beta, float *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_dsyrk_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const double *alpha, const double *A, rocblas_int lda, rocblas_stride stride_A, const double *beta, double *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_csyrk_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_zsyrk_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.6.4 syr2k 函数族

对称矩阵的双秩更新 C ：= alpha*(op(A)*op(B)^T + op(B)*op(A)^T) + beta*C。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) trans：指定对矩阵 A 和 B 是否进行转置；
4) n：矩阵 C 的行数和列数，即 C 是 n × n 的对称或埃尔米特矩阵；
5) k：op(A) 和 op(B) 的列数；
6) alpha：指向标量 alpha 的设备指针或主机指针；
7) A： 指向 GPU 上存储的矩阵 A 的指针；
8) lda： 指定 A 的主维度，每一列在内存中的跨度；
9) B：指向 GPU 上存储的矩阵 B 的指针；
10) ldb：指定 B 的主维度，每一列在内存中的跨度；

11) beta：指向标量 beta 的设备指针或主机指针；

12) C：C 矩阵指针,矩阵 C 的维度是 n × n；

13) ldc： 指定 C 的主维度，每一列在内存中的跨度；

14) batch_count：批次数；

15) strida_A：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；

16) strida_B：从一个 B_i 矩阵的开始到下一个 B_(i + 1) 的步幅；

17) strida_C：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_ssyr2k(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const float *alpha, const float *A, rocblas_int lda, const float *B, rocblas_int ldb, const float *beta, float *C, rocblas_int ldc)

rocblas_status rocblas_dsyr2k(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const double *alpha, const double *A, rocblas_int lda, const double *B, rocblas_int ldb, const double *beta, double *C, rocblas_int ldc)

rocblas_status rocblas_csyr2k(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *B, rocblas_int ldb, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zsyr2k(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *B, rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_ssyr2k_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const float *alpha, const float *const A[], rocblas_int lda, const float *const B[], rocblas_int ldb, const float *beta, float *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_dsyr2k_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const double *alpha, const double *const A[], rocblas_int lda, const double *const B[], rocblas_int ldb, const double *beta, double *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_csyr2k_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const B[], rocblas_int ldb, const rocblas_float_complex *beta, rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zsyr2k_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const B[], rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_ssyr2k_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const float *alpha, const float *A, rocblas_int lda, rocblas_stride stride_A, const float *B, rocblas_int ldb, rocblas_stride stride_B, const float *beta, float *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_dsyr2k_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const double *alpha, const double *A, rocblas_int lda, rocblas_stride stride_A, const double *B, rocblas_int ldb, rocblas_stride stride_B, const double *beta, double *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_csyr2k_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *B, rocblas_int ldb, rocblas_stride stride_B, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc,

rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_zsyr2k_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_B, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.6.5 syrkx 函数族

用于执行对称矩阵的 rank-k 更新。C ：= alpha*op(A)*op(B)^T + beta*C。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”
3) trans：指定对矩阵 A 和 B 是否进行转置；
4) n：矩阵 C 的行数和列数，即 C 是 n × n 的对称或埃尔米特矩阵；
5) k：op(A) 和 op(B) 的列数；
6) alpha：指向标量 alpha 的设备指针或主机指针；
7) A： 指向 GPU 上存储的矩阵 A 的指针；
8) lda： 指定 A 的主维度，每一列在内存中的跨度；
9) B：指向 GPU 上存储的矩阵 B 的指针；
10) ldb：指定 B 的主维度，每一列在内存中的跨度；
11) beta：指向标量 beta 的设备指针或主机指针；
12) C：C 矩阵指针,矩阵 C 的维度是 n × n；
13) ldc： 指定 C 的主维度，每一列在内存中的跨度；
14) batch_count：批次数；
15) strida_A：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；
16) strida_B：从一个 B_i 矩阵的开始到下一个 B_(i + 1) 的步幅；
17) strida_C：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_ssyrkx(rocblas_handle handle, rocblas_fill uplo,

rocblas_operation trans, rocblas_int n, rocblas_int k, const float *alpha, const float *A, rocblas_int lda, const float *B, rocblas_int ldb, const float *beta, float *C, rocblas_int ldc)

rocblas_status rocblas_dsyrkx(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const double *alpha, const double *A, rocblas_int lda, const double *B, rocblas_int ldb, const double *beta, double *C, rocblas_int ldc)

rocblas_status rocblas_csyrkx(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *B, rocblas_int ldb, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zsyrkx(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *B, rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_ssyrkx_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const float *alpha, const float *const A[], rocblas_int lda, const float *const B[], rocblas_int ldb, const float *beta, float *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_dsyrkx_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const double *alpha, const double *const A[], rocblas_int lda, const double *const B[], rocblas_int ldb, const double *beta, double *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_csyrkx_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const B[], rocblas_int ldb, const rocblas_float_complex *beta,

rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zsyrkx_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const B[], rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_ssyrkx_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const float *alpha, const float *A, rocblas_int lda, rocblas_stride stride_A, const float *B, rocblas_int ldb, rocblas_stride stride_B, const float *beta, float *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_dsyrkx_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const double *alpha, const double *A, rocblas_int lda, rocblas_stride stride_A, const double *B, rocblas_int ldb, rocblas_stride stride_B, const double *beta, double *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_csyrkx_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *B, rocblas_int ldb, rocblas_stride stride_B, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_zsyrkx_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_B, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.6.6 trmm 函数族

三角矩阵与一般矩阵乘法 C ：= alpha*op(A)*B, or C ：= alpha*B*op(A)。

参数：

1) handle：上下文队列的句柄；
2) side：指定三角矩阵 A 在乘法中的位置“rocblas_side_left”和“rocblas_side_right”；
3) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
4) transA：对矩阵 A 是否进行转置；
5) diag：指定 A 是否为单位三角矩阵，“rocblas_diagonal_unit”A 的主对角线被认为仅由 1 组成，并且未被引用，“rocblas_diagonal_non_unit”没有对 A 的主对角线做出任何假设；
6) m：B 的行数；
7) n：B 的列数；
8) alpha：指向标量 alpha 的设备指针或主机指针；
9) A： 指向 GPU 上存储的矩阵 A 的指针；
10) lda：指定 A 的主维度；
11) B：B 矩阵指针；
12) ldb：指定 B 的主维度；
13) C：C 矩阵指针；
14) ldc：指定 C 的主维度；
15) batch_count：批次数；
16) strida_A：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；
17) strida_B：从一个 B_i 矩阵的开始到下一个 B_(i + 1) 的步幅；
18) strida_C：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_strmm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, const float *B, rocblas_int ldb, float *C, rocblas_int ldc)

rocblas_status rocblas_dtrmm(rocblas_handle handle, rocblas_side side, rocblas_fill

uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, const double *B, rocblas_int ldb, double *C, rocblas_int ldc)

rocblas_status rocblas_ctrmm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *B, rocblas_int ldb, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_ztrmm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *B, rocblas_int ldb, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_strmm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const float *alpha, const float *const A[], rocblas_int lda, const float *const B[], rocblas_int ldb, float *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_dtrmm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const double *alpha, const double *const A[], rocblas_int lda, const double *const B[], rocblas_int ldb, double *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_ctrmm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const B[], rocblas_int ldb, rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_ztrmm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m,

rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const B[], rocblas_int ldb, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_strmm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, rocblas_stride stride_A, const float *B, rocblas_int ldb, rocblas_stride stride_B, float *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_dtrmm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, rocblas_stride stride_A, const double *B, rocblas_int ldb, rocblas_stride stride_B, double *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_ctrmm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *B, rocblas_int ldb, rocblas_stride stride_B, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_ztrmm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_B, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.6.7 trsm 函数族

解三角矩阵线性方程组 op(A)*X = alpha*B or X*op(A) = alpha*B,

用于求解系统。

参数：

1) handle：上下文队列的句柄；
2) side：指定三角矩阵 A 在乘法中的位置“rocblas_side_left”和“rocblas_side_right”；
3) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
4) transA：对矩阵 A 是否进行转置；
5) diag：指定 A 是否为单位三角矩阵，“rocblas_diagonal_unit”A 的主对角线被认为仅由 1 组成，并且未被引用，“rocblas_diagonal_non_unit”没有对 A 的主对角线做出任何假设；
6) m：B 的行数；
7) n：B 的列数；
8) alpha：指向标量 alpha 的设备指针或主机指针；
9) A： 指向 GPU 上存储的矩阵 A 的指针；
10) lda：指定 A 的主维度；
11) B：B 矩阵指针；
12) ldb：指定 B 的主维度；
13) batch_count：批次数；
14) strida_a：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；
15) strida_b：从一个 B_i 矩阵的开始到下一个 B_(i + 1) 的步幅；

rocblas_status rocblas_strsm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, float *B, rocblas_int ldb)

rocblas_status rocblas_dtrsm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, double *B, rocblas_int ldb)

rocblas_status rocblas_ctrsm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_float_complex *B, rocblas_int ldb)

rocblas_status rocblas_ztrsm(rocblas_handle handle, rocblas_side side, rocblas_fill

开发者社区

uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_double_complex *B, rocblas_int ldb)

rocblas_status rocblas_strsm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const float *alpha, const float *const A[], rocblas_int lda, float *const B[], rocblas_int ldb, rocblas_int batch_count)

rocblas_status rocblas_dtrsm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const double *alpha, const double *const A[], rocblas_int lda, double *const B[], rocblas_int ldb, rocblas_int batch_count)

rocblas_status rocblas_ctrsm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, rocblas_float_complex *const B[], rocblas_int ldb, rocblas_int batch_count)

rocblas_status rocblas_ztrsm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, rocblas_double_complex *const B[], rocblas_int ldb, rocblas_int batch_count)

rocblas_status rocblas_strsm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, rocblas_stride stride_a, float *B, rocblas_int ldb, rocblas_stride stride_b, rocblas_int batch_count)

rocblas_status rocblas_dtrsm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const double *alpha, const double *A, rocblas_int lda, rocblas_stride stride_a, double *B, rocblas_int ldb, rocblas_stride stride_b, rocblas_int batch_count)

rocblas_status rocblas_ctrsm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_a, rocblas_float_complex *B, rocblas_int ldb, rocblas_stride stride_b, rocblas_int batch_count)

rocblas_status rocblas_ztrsm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_a, rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_b, rocblas_int batch_count)

5.6.8 hemm 函数族

Hermitian（共轭对称）矩阵与一般矩阵乘法，用于复数域计算：

C ：= alpha*A*B + beta*C if side == rocblas_side_left,

C ：= alpha*B*A + beta*C if side == rocblas_side_right。

参数：

1) handle：上下文队列的句柄；
2) side：指定三角矩阵 A 在乘法中的位置“rocblas_side_left”和“rocblas_side_right”；
3) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
4) m：指定矩阵 B 和 C 的行数；
5) n：指定矩阵 B 和 C 的列数；
6) alpha：指向标量 alpha 的设备指针或主机指针；
7) A：指向 GPU 上存储的矩阵 A 的指针；
8) lda：指定矩阵 A 的主维度；
9) B：指向 GPU 上存储的矩阵 B 的指针；
10) ldb：指定矩阵 B 的主维度；
11) C：指向 GPU 上存储的矩阵 C 的指针；
12) ldc：指定矩阵 C 的主维度；
13) batch_count：批次数；

14) strida_A：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；

15) strida_B：从一个 B_i 矩阵的开始到下一个 B_(i + 1) 的步幅；

16) strida_C：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_chemm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *B, rocblas_int ldb, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zhemm(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *B, rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_chemm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const B[], rocblas_int ldb, const rocblas_float_complex *beta, rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zhemm_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const B[], rocblas_int ldb, const rocblas_double_complex *beta, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_chemm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *B, rocblas_int ldb, rocblas_stride stride_B, const rocblas_float_complex *beta, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_zhemm_strided_batched(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_B, const rocblas_double_complex *beta, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.6.9 herk 函数族

Hermitian 矩阵的秩更新 C ：=alpha*op(A)*op(A)^H+beta*C，用于复数域优化。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”，指定矩阵 C 是上三角还是下三角；
3) transA：对矩阵 A 是否进行转置；
4) n：指定矩阵 C 的行数和列数；
5) k：指定 op(A) 的列数；
6) alpha：指向标量 alpha 的设备指针或主机指针；
7) A：指向 GPU 上存储的矩阵 A 的指针；
8) lda：指定矩阵 A 的主维度；
9) beta：指向标量 beta 的设备指针或主机指针；
10) C：指向 GPU 上存储的矩阵 C 的指针；
11) ldc: 指定矩阵 C 的主维度；
17) batch_count：批次数；
18) strida_A：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；
19) strida_C：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_cherk(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const float *alpha, const rocblas_float_complex *A, rocblas_int lda, const float *beta, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zherk(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const double *alpha, const rocblas_double_complex *A, rocblas_int lda, const double *beta, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_cherk_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const float *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const float *beta, rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zherk_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const double *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const double *beta, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_cherk_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const float *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const float *beta, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_zherk_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const double *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const double *beta, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.6.10 her2k 函数族

Hermitian 矩阵的双秩更新：

C ：= alpha*op(A)*op(B)^H + conj(alpha)*op(B)*op(A)^H + beta*C。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”，指定矩阵 C 是上三角还是下三角；

3) trans：对矩阵 A 是否进行转置；

4) n：指定矩阵 C 的行数和列数；

5) k：指定 op(A) 的列数；

6) alpha：指向标量 alpha 的设备指针或主机指针；

7) A：指向 GPU 上存储的矩阵 A 的指针；

8) lda：指定矩阵 A 的主维度；

9) B：指向 GPU 上存储的矩阵 B 的指针；

10) ldb：指定矩阵 B 的主维度；

11) beta：指向标量 beta 的设备指针或主机指针；

12) C：指向 GPU 上存储的矩阵 C 的指针；

13) ldc: 指定矩阵 C 的主维度；

20) batch_count：批次数；

21) strida_A：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；

22) strida_B：从一个 B_i 矩阵的开始到下一个 B_(i + 1) 的步幅；

23) strida_C：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_cher2k(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *B, rocblas_int ldb, const float *beta, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zher2k(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *B, rocblas_int ldb, const double *beta, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_cher2k_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const B[], rocblas_int ldb, const float *beta, rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zher2k_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const B[], rocblas_int ldb, const double *beta, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_cher2k_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *B, rocblas_int ldb, rocblas_stride stride_B, const float *beta, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_zher2k_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_B, const double *beta, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.6.11 herkx 函数族

复数域的 rank-k 更新 C ：= alpha*op(A)*op(B)^H + beta*C。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”，指定矩阵 C 是上三角还是下三角；
3) trans：矩阵是否进行转置；
4) n：指定矩阵 C 的行数和列数；
5) k：指定 op(A) 的列数；
6) alpha：指向标量 alpha 的设备指针或主机指针；
7) A：指向 GPU 上存储的矩阵 A 的指针；
8) lda：指定矩阵 A 的主维度；

9) B：指向 GPU 上存储的矩阵 B 的指针；

10) ldb：指定矩阵 B 的主维度；

11) beta：指向标量 beta 的设备指针或主机指针；

12) C：指向 GPU 上存储的矩阵 C 的指针；

13) ldc: 指定矩阵 C 的主维度；

24) batch_count：批次数；

25) strida_A：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；

26) strida_B：从一个 B_i 矩阵的开始到下一个 B_(i + 1) 的步幅；

27) strida_C：从一个 C_i 矩阵的开始到下一个 C_(i + 1) 的步幅；

rocblas_status rocblas_cherkx(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *B, rocblas_int ldb, const float *beta, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zherkx(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *B, rocblas_int ldb, const double *beta, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_cherkx_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_float_complex *const B[], rocblas_int ldb, const float *beta, rocblas_float_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zherkx_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas_double_complex *const B[], rocblas_int ldb, const double *beta, rocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_cherkx_strided_batched(rocblas_handle handle, rocblas_fill

uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_float_complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_float_complex *B, rocblas_int ldb, rocblas_stride stride_B, const float *beta, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_zherkx_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_operation trans, rocblas_int n, rocblas_int k, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_B, const double *beta, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.6.12 trtri 函数族

求解三角矩阵逆。

参数：

1) handle：上下文队列的句柄；
2) uplo：rocblas_fill 指定上部“rocblas_fill_upper”或下部“rocblas_fill_lower”；
3) diag：指定 A 是否为单位三角矩阵，“rocblas_diagonal_unit”A 的主对角线被认为仅由 1 组成，并且未被引用，“rocblas_diagonal_non_unit”没有对 A 的主对角线做出任何假设；
4) n：矩阵 A 和 invA 的大小（n × n）；
5) A：指向设备端矩阵 A 的指针；
6) lda：指定矩阵 A 的主维度；
7) invA：指向设备端矩阵 invA 的指针，用于存储 A 的逆矩阵；
8) IdinvA：指定矩阵 invA 的主维度；
9) batch_count：批次数；
10) strida_a：从一个 A_i 矩阵的开始到下一个 A_(i + 1) 的步幅；

rocblas_status rocblas_strtri(rocblas_handle handle, rocblas_fill uplo, rocblas_diagonal diag, rocblas_int n, const float *A, rocblas_int lda, float *invA, rocblas_int ldinvA)

rocblas_status rocblas_dtrtri(rocblas_handle handle, rocblas_fill uplo,

rocblas_diagonal diag, rocblas_int n, const double *A, rocblas_int lda, double *invA, rocblas_int ldinvA)

rocblas_status rocblas_strtri_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_diagonal diag, rocblas_int n, const float *const A[], rocblas_int lda, float *const invA[], rocblas_int ldinvA, rocblas_int batch_count)

rocblas_status rocblas_dtrtri_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_diagonal diag, rocblas_int n, const double *const A[], rocblas_int lda, double *const invA[], rocblas_int ldinvA, rocblas_int batch_count)

rocblas_status rocblas_strtri_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_diagonal diag, rocblas_int n, const float *A, rocblas_int lda, rocblas_stride stride_a, float *invA, rocblas_int ldinvA, rocblas_stride stride_invA, rocblas_int batch_count)

rocblas_status rocblas_dtrtri_strided_batched(rocblas_handle handle, rocblas_fill uplo, rocblas_diagonal diag, rocblas_int n, const double *A, rocblas_int lda, rocblas_stride stride_a, double *invA, rocblas_int ldinvA, rocblas_stride stride_invA, rocblas_int batch_count)

5.7 rocBLAS 扩展函数

rocBLAS 的扩展接口（Extended API）是对传统 BLAS 接口的增强版，主要目标是提升灵活性、支持混合精度计算、适应 AI / HPC 应用中的复杂需求。

5.7.1 Axpy

支持类型如下：

alpha_type	x_type	y_type	execution_type
bf16_r	bf16_r	bf16_r	f32_r
f32_r	bf16_r	bf16_r	f32_r
f16_r	f16_r	f16_r	f16_r
f16_r	f16_r	f16_r	f32_r
f32_r	f16_r	f16_r	f32_r
f32_r	f32_r	f32_r	f32_r
f64_r	f64_r	f64_r	f64_r
f32_c	f32_c	f32_c	f32_c
f64_c	f64_c	f64_c	f64_c

rocblas_status rocblas_axpy_ex(rocblas_handle handle, rocblas_int n, const void *alpha, rocblas_datatype alpha_type, const void *x, rocblas_datatype x_type, rocblas_int incx, void *y, rocblas_datatype y_type, rocblas_int incy, rocblas_datatype execution_type)

rocblas_status rocblas_axpy_batched_ex(rocblas_handle handle, rocblas_int n, const void *alpha, rocblas_datatype alpha_type, const void *x, rocblas_datatype x_type, rocblas_int incx, void *y, rocblas_datatype y_type, rocblas_int incy, rocblas_int batch_count, rocblas_datatype execution_type)

rocblas_status rocblas_axpy_strided_batched_ex(rocblas_handle handle, rocblas_int n, const void *alpha, rocblas_datatype alpha_type, const void *x, rocblas_datatype x_type, rocblas_int incx, rocblas_stride stridex, void *y, rocblas_datatype y_type, rocblas_int incy, rocblas_stride stridey, rocblas_int batch_count, rocblas_datatype execution_type)

5.7.2 Dot

支持类型如下：

x_type	y_type	result_type	execution_type
f16_r	f16_r	f16_r	f16_r
f16_r	f16_r	f16_r	f32_r
bf16_r	bf16_r	bf16_r	f32_r
f32_r	f32_r	f32_r	f32_r
f64_r	f64_r	f64_r	f64_r
f32_c	f32_c	f32_c	f32_c
f64_c	f64_c	f64_c	f64_c

rocblas_status rocblas_dot_ex(rocblas_handle handle, rocblas_int n, const void *x, rocblas_datatype x_type, rocblas_int incx, const void *y, rocblas_datatype y_type, rocblas_int incy, void *result, rocblas_datatype result_type, rocblas_datatype execution_type)

rocblas_status rocblas_dot_batched_ex(rocblas_handle handle, rocblas_int n, const void *x, rocblas_datatype x_type, rocblas_int incx, const void *y, rocblas_datatype y_type, rocblas_int incy, rocblas_int batch_count, void *result, rocblas_datatype result_type, rocblas_datatype execution_type)

rocblas_status rocblas_dot_strided_batched_ex(rocblas_handle handle, rocblas_int n, const void *x, rocblas_datatype x_type, rocblas_int incx, rocblas_stride stride_x, const void *y, rocblas_datatype y_type, rocblas_int incy, rocblas_stride stride_y, rocblas_int batch_count, void *result, rocblas_datatype result_type, rocblas_datatype execution_type)

5.7.3 Dotc

支持类型如下：

x_type	y_type	result_type	execution_type
f16_r	f16_r	f16_r	f16_r
f16_r	f16_r	f16_r	f32_r
bf16_r	bf16_r	bf16_r	f32_r
f32_r	f32_r	f32_r	f32_r
f64_r	f64_r	f64_r	f64_r
f32_c	f32_c	f32_c	f32_c
f64_c	f64_c	f64_c	f64_c

rocblas_status rocblas_dotc_ex(rocblas_handle handle, rocblas_int n, const void *x, r
ocblas_datatype x_type, rocblas_int incx, const void *y, rocblas_datatype y_type, roc
blas_int incy, void *result, rocblas_datatype result_type, rocblas_datatype execution_t
ype)

rocblas_status rocblas_dotc_batched_ex(rocblas_handle handle, rocblas_int n, const v
oid *x, rocblas_datatype x_type, rocblas_int incx, const void *y, rocblas_datatype y_t
ype, rocblas_int incy, rocblas_int batch_count, void *result, rocblas_datatype result_t
ype, rocblas_datatype execution_type)

rocblas_status rocblas_dotc_strided_batched_ex(rocblas_handle handle, rocblas_int n,
const void *x, rocblas_datatype x_type, rocblas_int incx, rocblas_stride stride_x, con
st void *y, rocblas_datatype y_type, rocblas_int incy, rocblas_stride stride_y, rocblas_
int batch_count, void *result, rocblas_datatype result_type, rocblas_datatype executio
n_type)

5.7.4 Nrm2

支持类型如下：

x_type	result	execution_type
bf16_r	bf16_r	f32_r
f16_r	f16_r	f32_r
f32_r	f32_r	f32_r
f64_r	f64_r	f64_r
f32_c	f32_r	f32_r
f64_c	f64_r	f64_r

rocblas_status rocblas_nrm2_ex(rocblas_handle handle, rocblas_int n, const void *x, r ocblas_datatype x_type, rocblas_int incx, void *results, rocblas_datatype result_type, rocblas_datatype execution_type)

rocblas_status rocblas_nrm2_batched_ex(rocblas_handle handle, rocblas_int n, const void *x, rocblas_datatype x_type, rocblas_int incx, rocblas_int batch_count, void *res ults, rocblas_datatype result_type, rocblas_datatype execution_type)

rocblas_status rocblas_nrm2_strided_batched_ex(rocblas_handle handle, rocblas_int n, const void *x, rocblas_datatype x_type, rocblas_int incx, rocblas_stride stride_x, ro cblas_int batch_count, void *results, rocblas_datatype result_type, rocblas_datatype e xecution_type)

5.7.5 Rot

支持类型如下：

x_type	y_type	cs_type	execution_type
bf16_r	bf16_r	bf16_r	f32_r
f16_r	f16_r	f16_r	f32_r
f32_r	f32_r	f32_r	f32_r
f64_r	f64_r	f64_r	f64_r
f32_c	f32_c	f32_c	f32_c
f32_c	f32_c	f32_r	f32_c
f64_c	f64_c	f64_c	f64_c
f64_c	f64_c	f64_r	f64_c

rocblas_status rocblas_rot_ex(rocblas_handle handle, rocblas_int n, void *x, rocblas_datatype x_type, rocblas_int incx, void *y, rocblas_datatype y_type, rocblas_int incy, const void *c, const void *s, rocblas_datatype cs_type, rocblas_datatype execution_type)

rocblas_status rocblas_rot_batched_ex(rocblas_handle handle, rocblas_int n, void *x, rocblas_datatype x_type, rocblas_int incx, void *y, rocblas_datatype y_type, rocblas_int incy, const void *c, const void *s, rocblas_datatype cs_type, rocblas_int batch_count, rocblas_datatype execution_type)

rocblas_status rocblas_rot_strided_batched_ex(rocblas_handle handle, rocblas_int n, void *x, rocblas_datatype x_type, rocblas_int incx, rocblas_stride stride_x, void *y, rocblas_datatype y_type, rocblas_int incy, rocblas_stride stride_y, const void *c, const void *s, rocblas_datatype cs_type, rocblas_int batch_count, rocblas_datatype execution_type)

5.7.6 Scal

支持类型如下：

alpha_type	x_type	execution_type
f32_r	bf16_r	f32_r
bf16_r	bf16_r	f32_r
f16_r	f16_r	f16_r
f16_r	f16_r	f32_r
f32_r	f16_r	f32_r
f32_r	f32_r	f32_r
f64_r	f64_r	f64_r
f32_c	f32_c	f32_c
f64_c	f64_c	f64_c
f32_r	f32_c	f32_c
f64_r	f64_c	f64_c

rocblas_status rocblas_scal_ex(rocblas_handle handle, rocblas_int n, const void *alph

a, rocblas_datatype alpha_type, void *x, rocblas_datatype x_type, rocblas_int incx, ro
cblas_datatype execution_type)

rocblas_status rocblas_scal_batched_ex(rocblas_handle handle, rocblas_int n, const v
oid *alpha, rocblas_datatype alpha_type, void *x, rocblas_datatype x_type, rocblas_in
t incx, rocblas_int batch_count, rocblas_datatype execution_type)

rocblas_status rocblas_scal_strided_batched_ex(rocblas_handle handle, rocblas_int n,
const void *alpha, rocblas_datatype alpha_type, void *x, rocblas_datatype x_type, ro
cblas_int incx, rocblas_stride stridex, rocblas_int batch_count, rocblas_datatype execu
tion_type)

5.7.7 Gemm

支持类型如下：

1) rocblas_datatype_f64_r = a_type = b_type = c_type = d_type = compute_type
2) rocblas_datatype_f32_r = a_type = b_type = c_type = d_type = compute_type
3) rocblas_datatype_f16_r = a_type = b_type = c_type = d_type = compute_type
4) rocblas_datatype_f16_r = a_type = b_type = c_type = d_type; rocblas_datatype_f32_r = compute_type
5) rocblas_datatype_f16_r = a_type = b_type; rocblas_datatype_f32_r = c_type = d_type = compute_type
6) rocblas_datatype_bf16_r = a_type = b_type = c_type = d_type; rocblas_datatype_f32_r = compute_type
7) rocblas_datatype_bf16_r = a_type = b_type; rocblas_datatype_f32_r = c_type = d_type = compute_type
8) rocblas_datatype_i8_r = a_type = b_type; rocblas_datatype_i32_r = c_type = d_type = compute_type
9) rocblas_datatype_f32_c = a_type = b_type = c_type = d_type = compute_type
10) rocblas_datatype_f64_c = a_type = b_type = c_type = d_type = compute_type

rocblas_status rocblas_gemm_ex(rocblas_handle handle, rocblas_operation transA, ro
cblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const void *alpha,
const void *a, rocblas_datatype a_type, rocblas_int lda, const void *b, rocblas_dataty

pe b_type, rocblas_int ldb, const void *beta, const void *c, rocblas_datatype c_type, r
ocblas_int ldc, void *d, rocblas_datatype d_type, rocblas_int ldd, rocblas_datatype co
mpute_type, rocblas_gemm_algo algo, int32_t solution_index, uint32_t flags)

rocblas_status rocblas_gemm_batched_ex(rocblas_handle handle, rocblas_operation t
ransA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, const voi
d *alpha, const void *a, rocblas_datatype a_type, rocblas_int lda, const void *b, rocbl
as_datatype b_type, rocblas_int ldb, const void *beta, const void *c, rocblas_datatype
c_type, rocblas_int ldc, void *d, rocblas_datatype d_type, rocblas_int ldd, rocblas_int
batch_count, rocblas_datatype compute_type, rocblas_gemm_algo algo, int32_t soluti
on_index, uint32_t flags)

rocblas_status rocblas_gemm_strided_batched_ex(rocblas_handle handle, rocblas_op
eration transA, rocblas_operation transB, rocblas_int m, rocblas_int n, rocblas_int k, c
onst void *alpha, const void *a, rocblas_datatype a_type, rocblas_int lda, rocblas_stri
de stride_a, const void *b, rocblas_datatype b_type, rocblas_int ldb, rocblas_stride stri
de_b, const void *beta, const void *c, rocblas_datatype c_type, rocblas_int ldc, rocbla
s_stride stride_c, void *d, rocblas_datatype d_type, rocblas_int ldd, rocblas_stride stri
de_d, rocblas_int batch_count, rocblas_datatype compute_type, rocblas_gemm_algo a
lgo, int32_t solution_index, uint32_t flags)

5.7.8 Trsm

支持类型如下参考 gemm_ex 支持类型。

rocblas_status rocblas_trsm_ex(rocblas_handle handle, rocblas_side side, rocblas_fill
uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas_int n, c
onst void *alpha, const void *A, rocblas_int lda, void *B, rocblas_int ldb, const void
*invA, rocblas_int invA_size, rocblas_datatype compute_type)

rocblas_status rocblas_trsm_batched_ex(rocblas_handle handle, rocblas_side side, roc
blas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m, rocblas
_int n, const void *alpha, const void *A, rocblas_int lda, void *B, rocblas_int ldb, roc
blas_int batch_count, const void *invA, rocblas_int invA_size, rocblas_datatype com
pute_type)

rocblas_status rocblas_trsm_strided_batched_ex(rocblas_handle handle, rocblas_side side, rocblas_fill uplo, rocblas_operation transA, rocblas_diagonal diag, rocblas_int m , rocblas_int n, const void *alpha, const void *A, rocblas_int lda, rocblas_stride stride _A, void *B, rocblas_int ldb, rocblas_stride stride_B, rocblas_int batch_count, const v oid *invA, rocblas_int invA_size, rocblas_stride stride_invA, rocblas_datatype compu te_type)

5.7.9 Geam

geam 执行矩阵加法,支持类型和参数参考 gemm_ex。

rocblas_status rocblas_sgeam(rocblas_handle handle, rocblas_operation transA, rocbl as_operation transB, rocblas_int m, rocblas_int n, const float *alpha, const float *A, r ocblas_int lda, const float *beta, const float *B, rocblas_int ldb, float *C, rocblas_int l dc)

rocblas_status rocblas_dgeam(rocblas_handle handle, rocblas_operation transA, rocbl as_operation transB, rocblas_int m, rocblas_int n, const double *alpha, const double * A, rocblas_int lda, const double *beta, const double *B, rocblas_int ldb, double *C, ro cblas_int ldc)

rocblas_status rocblas_cgeam(rocblas_handle handle, rocblas_operation transA, rocbl as_operation transB, rocblas_int m, rocblas_int n, const rocblas_float_complex *alpha , const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *beta, const rocblas_float_complex *B, rocblas_int ldb, rocblas_float_complex *C, rocblas _int ldc)

rocblas_status rocblas_zgeam(rocblas_handle handle, rocblas_operation transA, rocbl as_operation transB, rocblas_int m, rocblas_int n, const rocblas_double_complex *alp ha, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_comple x *beta, const rocblas_double_complex *B, rocblas_int ldb, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_sgeam_batched(rocblas_handle handle, rocblas_operation tran sA, rocblas_operation transB, rocblas_int m, rocblas_int n, const float *alpha, const fl oat *const A[], rocblas_int lda, const float *beta, const float *const B[], rocblas_int ld

b, float *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_dgeam_batched(rocblas_handle handle, rocblas_operation tran sA, rocblas_operation transB, rocblas_int m, rocblas_int n, const double *alpha, const double *const A[], rocblas_int lda, const double *beta, const double *const B[], rocb las_int ldb, double *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_cgeam_batched(rocblas_handle handle, rocblas_operation tran sA, rocblas_operation transB, rocblas_int m, rocblas_int n, const rocblas_float_compl ex *alpha, const rocblas_float_complex *const A[], rocblas_int lda, const rocblas_floa t_complex *beta, const rocblas_float_complex *const B[], rocblas_int ldb, rocblas_flo at_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_zgeam_batched(rocblas_handle handle, rocblas_operation tran sA, rocblas_operation transB, rocblas_int m, rocblas_int n, const rocblas_double_com plex *alpha, const rocblas_double_complex *const A[], rocblas_int lda, const rocblas _double_complex *beta, const rocblas_double_complex *const B[], rocblas_int ldb, r ocblas_double_complex *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_sgeam_strided_batched(rocblas_handle handle, rocblas_operat ion transA, rocblas_operation transB, rocblas_int m, rocblas_int n, const float *alpha, const float *A, rocblas_int lda, rocblas_stride stride_A, const float *beta, const float * B, rocblas_int ldb, rocblas_stride stride_B, float *C, rocblas_int ldc, rocblas_stride str ide_C, rocblas_int batch_count)

rocblas_status rocblas_dgeam_strided_batched(rocblas_handle handle, rocblas_operat ion transA, rocblas_operation transB, rocblas_int m, rocblas_int n, const double *alph a, const double *A, rocblas_int lda, rocblas_stride stride_A, const double *beta, const double *B, rocblas_int ldb, rocblas_stride stride_B, double *C, rocblas_int ldc, rocbla s_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_cgeam_strided_batched(rocblas_handle handle, rocblas_operat ion transA, rocblas_operation transB, rocblas_int m, rocblas_int n, const rocblas_float _complex *alpha, const rocblas_float_complex *A, rocblas_int lda, rocblas_stride stri de_A, const rocblas_float_complex *beta, const rocblas_float_complex *B, rocblas_i

nt ldb, rocblas_stride stride_B, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

rocblas_status rocblas_zgeam_strided_batched(rocblas_handle handle, rocblas_operation transA, rocblas_operation transB, rocblas_int m, rocblas_int n, const rocblas_double_complex *alpha, const rocblas_double_complex *A, rocblas_int lda, rocblas_stride stride_A, const rocblas_double_complex *beta, const rocblas_double_complex *B, rocblas_int ldb, rocblas_stride stride_B, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_int batch_count)

5.7.10 Sdgmm

sdgmm 执行矩阵与对角矩阵之间的乘法运算即：

1) C = A * diag(x) 当 side == rocblas_side_right

2) C = diag(x) * A 当 side == rocblas_side_left

其中，A 和 C 是维度 mxn 的矩阵，diag(x) 是由向量 x 构造的对角矩阵，x 的长度依赖于 side 参数。

rocblas_status rocblas_sdgmm(rocblas_handle handle, rocblas_side side, rocblas_int m, rocblas_int n, const float *A, rocblas_int lda, const float *x, rocblas_int incx, float *C, rocblas_int ldc)

rocblas_status rocblas_ddgmm(rocblas_handle handle, rocblas_side side, rocblas_int m, rocblas_int n, const double *A, rocblas_int lda, const double *x, rocblas_int incx, double *C, rocblas_int ldc)

rocblas_status rocblas_cdgmm(rocblas_handle handle, rocblas_side side, rocblas_int m, rocblas_int n, const rocblas_float_complex *A, rocblas_int lda, const rocblas_float_complex *x, rocblas_int incx, rocblas_float_complex *C, rocblas_int ldc)

rocblas_status rocblas_zdgmm(rocblas_handle handle, rocblas_side side, rocblas_int m, rocblas_int n, const rocblas_double_complex *A, rocblas_int lda, const rocblas_double_complex *x, rocblas_int incx, rocblas_double_complex *C, rocblas_int ldc)

rocblas_status rocblas_sdgmm_batched(rocblas_handle handle, rocblas_side side, rocblas_int m, rocblas_int n, const float *const A[], rocblas_int lda, const float *const x[], rocblas_int incx, float *const C[], rocblas_int ldc, rocblas_int batch_count)

rocblas_status rocblas_ddgmm_batched(rocblas_handle handle, rocblas_side side, roc
blas_int m, rocblas_int n, const double *const A[], rocblas_int lda, const double *cons
t x[], rocblas_int incx, double *const C[], rocblas_int ldc, rocblas_int batch_count)
rocblas_status rocblas_cdgmm_batched(rocblas_handle handle, rocblas_side side, roc
blas_int m, rocblas_int n, const rocblas_float_complex *const A[], rocblas_int lda, co
nst rocblas_float_complex *const x[], rocblas_int incx, rocblas_float_complex *const
C[], rocblas_int ldc, rocblas_int batch_count)
rocblas_status rocblas_zdgmm_batched(rocblas_handle handle, rocblas_side side, roc
blas_int m, rocblas_int n, const rocblas_double_complex *const A[], rocblas_int lda,
const rocblas_double_complex *const x[], rocblas_int incx, rocblas_double_complex
*const C[], rocblas_int ldc, rocblas_int batch_count)
rocblas_status rocblas_sdgmm_strided_batched(rocblas_handle handle, rocblas_side s
ide, rocblas_int m, rocblas_int n, const float *A, rocblas_int lda, rocblas_stride stride_
A, const float *x, rocblas_int incx, rocblas_stride stride_x, float *C, rocblas_int ldc, r
ocblas_stride stride_C, rocblas_int batch_count)
rocblas_status rocblas_ddgmm_strided_batched(rocblas_handle handle, rocblas_side
side, rocblas_int m, rocblas_int n, const double *A, rocblas_int lda, rocblas_stride stri
de_A, const double *x, rocblas_int incx, rocblas_stride stride_x, double *C, rocblas_i
nt ldc, rocblas_stride stride_C, rocblas_int batch_count)
rocblas_status rocblas_cdgmm_strided_batched(rocblas_handle handle, rocblas_side s
ide, rocblas_int m, rocblas_int n, const rocblas_float_complex *A, rocblas_int lda, roc
blas_stride stride_A, const rocblas_float_complex *x, rocblas_int incx, rocblas_stride
stride_x, rocblas_float_complex *C, rocblas_int ldc, rocblas_stride stride_C, rocblas_
int batch_count)
rocblas_status rocblas_zdgmm_strided_batched(rocblas_handle handle, rocblas_side s
ide, rocblas_int m, rocblas_int n, const rocblas_double_complex *A, rocblas_int lda, r
ocblas_stride stride_A, const rocblas_double_complex *x, rocblas_int incx, rocblas_st
ride stride_x, rocblas_double_complex *C, rocblas_int ldc, rocblas_stride stride_C, ro
cblas_int batch_count)

5.7.11 Syrk_Ex

syrk_ex 是对称矩阵 rank-k 更新的扩展接口，计算 C := alpha * op(A) * op(A)^T + beta * C。C 为 n × n 对称矩阵，只访问由 uplo 指定的上三角或下三角部分。该接口通过 a_type、c_type 和 execution_type 显式指定 A、C 和计算精度，适用于输入、输出与计算精度不同的场景。

支持的组合：bf16_r → bf16_r/f32_r（f32_r 计算）；f16_r → f16_r/f32_r（f32_r 计算）；f32_r → f32_r/f64_r（f64_r 计算）。

参数说明：uplo 选择 C 的有效三角部分；transA 选择 op(A)=A 或 A^T；n 为 C 的阶数；k 为 op(A) 的列数；lda 与 ldc 分别为 A、C 的首维。alpha 和 beta 的数据类型须与 execution_type 一致。

注意：rocblas_syrk_ex 提供 C 和 Fortran 接口；本版本不提供 ILP64（_64 后缀）接口。

```
rocblas_status rocblas_syrk_ex(rocblas_handle handle, rocblas_fill uplo, rocblas_operation transA, rocblas_int n, rocblas_int k, const void* alpha, const void* A, rocblas_datatype a_type, rocblas_int lda, const void* beta, void* C, rocblas_datatype c_type, rocblas_int ldc, rocblas_datatype execution_type);
```

6 rocBLAS 基准测试

6.1 rocblas-bench

rocblas-bench 用于衡量性能并验证 rocBLAS 函数的正确性。构建 rocBLAS 客户端后，可以在目录中找到它们 rocBLAS/build/release/clients/staging，或者直接在 DTK 中查找此工具。

它有一个命令行界面：

```
rocBLAS/build/release/clients/staging/rocblas-bench --help
```

非 HPA 案例（gemm）：

```
#dgemm
$ ./rocblas-bench -f gemm --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r d --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1.0
# dgemm batched
```

```
$ ./rocblas-bench -f gemm_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r d --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1 --batch_count 5
# dgemm strided batched
$ ./rocblas-bench -f gemm_strided_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r d --lda 1024 --stride_a 4096 --ldb 2048 --stride_b 4096 --ldc 1024 --stride_c 2097152 --ldd 1024 --stride_d 2097152 --alpha 1.1 --beta 1 --batch_count 5

# sgemm
$ ./rocblas-bench -f gemm --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r s --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1
# sgemm batched
$ ./rocblas-bench -f gemm_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r s --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1 --batch_count 5
# sgemm strided batched
$ ./rocblas-bench -f gemm_strided_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r s --lda 1024 --stride_a 4096 --ldb 2048 --stride_b 4096 --ldc 1024 --stride_c 2097152 --ldd 1024 --stride_d 2097152 --alpha 1.1 --beta 1 --batch_count 5

# hgemm (this function is not really very fast. Use HHS instead, which is faster and more accurate)
$ ./rocblas-bench -f gemm --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r h --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1
# hgemm batched
$ ./rocblas-bench -f gemm_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r h --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1 --batch_count 5
# hgemm strided batched
```

```
$ ./rocblas-bench -f gemm_strided_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r h --lda 1024 --stride_a 4096 --ldb 2048 --stride_b 4096 --ldc 1024 --stride_c 2097152 --ldd 1024 --stride_d 2097152 --alpha 1.1 --beta 1 --batch_count 5

# cgemm
$ ./rocblas-bench -f gemm --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r c --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1
# cgemm batched
$ ./rocblas-bench -f gemm_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r c --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1 --batch_count 5
# cgemm strided batched
$ ./rocblas-bench -f gemm_strided_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r c --lda 1024 --stride_a 4096 --ldb 2048 --stride_b 4096 --ldc 1024 --stride_c 2097152 --ldd 1024 --stride_d 2097152 --alpha 1.1 --beta 1 --batch_count 5

# zgemm
$ ./rocblas-bench -f gemm --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r z --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1
# zgemm batched
$ ./rocblas-bench -f gemm_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r z --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1 --batch_count 5
# zgemm strided batched
$ ./rocblas-bench -f gemm_strided_batched --transposeA N --transposeB T -m 1024 -n 2048 -k 512 -r z --lda 1024 --stride_a 4096 --ldb 2048 --stride_b 4096 --ldc 1024 --stride_c 2097152 --ldd 1024 --stride_d 2097152 --alpha 1.1 --beta 1 --batch_count 5

# cgemm (NC)
```

```
$ ./rocblas-bench -f gemm --transposeA N --transposeB C -m 1024 -n 2048 -k 512 -r c
--lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1
# cgemm batched (NC)
$ ./rocblas-bench -f gemm_batched --transposeA N --transposeB C -m 1024 -n 2048 -
k 512 -r c --lda 1024 --ldb 2048 --ldc 1024 --ldd 1024 --alpha 1.1 --beta 1 --batch_count
5
# cgemm strided batched (NC)
$ ./rocblas-bench -f gemm_strided_batched --transposeA N --transposeB C -m 1024 -n
2048 -k 512 -r c --lda 1024 --stride_a 4096 --ldb 2048 --stride_b 4096 --ldc 1024 --
stride_c 2097152 --ldd 1024 --stride_d 2097152 --alpha 1.1 --beta 1 --batch_count 5
```

HPA 案例（gemm_ex）

```
# HHS
$ ./rocblas-bench -f gemm_ex --transposeA N --transposeB T -m 1024 -n 2048 -k 512
--a_type h --lda 1024 --b_type h --ldb 2048 --c_type h --ldc 1024 --d_type h --ldd 1024
--compute_type s --alpha 1.1 --beta 1
# HHS batched
$ ./rocblas-bench -f gemm_batched_ex --transposeA N --transposeB T -m 1024 -n 2048
-k 512 --a_type h --lda 1024 --b_type h --ldb 2048 --c_type h --ldc 1024 --d_type h --
ldd 1024 --compute_type s --alpha 1.1 --beta 1 --batch_count 5
# HHS strided batched
$ ./rocblas-bench -f gemm_strided_batched_ex --transposeA N --transposeB T -m 1024
-n 2048 -k 512 --a_type h --lda 1024 --stride_a 4096 --b_type h --ldb 2048 --stride_b
4096 --c_type h --ldc 1024 --stride_c 2097152 --d_type h --ldd 1024 --stride_d 2097152
--compute_type s --alpha 1.1 --beta 1 --batch_count 5


# HSS
$ ./rocblas-bench -f gemm_ex --transposeA N --transposeB T -m 1024 -n 2048 -k 512
--a_type h --lda 1024 --b_type h --ldb 2048 --c_type s --ldc 1024 --d_type s --ldd 1024
--compute_type s --alpha 1.1 --beta 1
```

```
# HSS batched
$ ./rocblas-bench -f gemm_batched_ex --transposeA N --transposeB T -m 1024 -n 2048
-k 512 --a_type h --lda 1024 --b_type h --ldb 2048 --c_type s --ldc 1024 --d_type s --
ldd 1024 --compute_type s --alpha 1.1 --beta 1 --batch_count 5
# HSS strided batched
$ ./rocblas-bench -f gemm_strided_batched_ex --transposeA N --transposeB T -m 1024
-n 2048 -k 512 --a_type h --lda 1024 --stride_a 4096 --b_type h --ldb 2048 --stride_b
4096 --c_type s --ldc 1024 --stride_c 2097152 --d_type s --ldd 1024 --stride_d 2097152
--compute_type s --alpha 1.1 --beta 1 --batch_count 5

# BBS
$ ./rocblas-bench -f gemm_ex --transposeA N --transposeB T -m 1024 -n 2048 -k 512
--a_type bf16_r --lda 1024 --b_type bf16_r --ldb 2048 --c_type bf16_r --ldc 1024 --
d_type bf16_r --ldd 1024 --compute_type s --alpha 1.1 --beta 1
# BBS batched
$ ./rocblas-bench -f gemm_batched_ex --transposeA N --transposeB T -m 1024 -n 2048
-k 512 --a_type bf16_r --lda 1024 --b_type bf16_r --ldb 2048 --c_type bf16_r --ldc 1024
--d_type bf16_r --ldd 1024 --compute_type s --alpha 1.1 --beta 1 --batch_count 5
# BBS strided batched
$ ./rocblas-bench -f gemm_strided_batched_ex --transposeA N --transposeB T -m 1024
-n 2048 -k 512 --a_type bf16_r --lda 1024 --stride_a 4096 --b_type bf16_r --ldb 2048 -
-stride_b 4096 --c_type bf16_r --ldc 1024 --stride_c 2097152 --d_type bf16_r --ldd
1024 --stride_d 2097152 --compute_type s --alpha 1.1 --beta 1 --batch_count 5

# BSS
$ ./rocblas-bench -f gemm_ex --transposeA N --transposeB T -m 1024 -n 2048 -k 512
--a_type bf16_r --lda 1024 --b_type bf16_r --ldb 2048 --c_type s --ldc 1024 --d_type s
--ldd 1024 --compute_type s --alpha 1.1 --beta 1
# BSS batched
```

```
$ ./rocblas-bench -f gemm_batched_ex --transposeA N --transposeB T -m 1024 -n 2048 -k 512 --a_type bf16_r --lda 1024 --b_type bf16_r --ldb 2048 --c_type s --ldc 1024 --d_type s --ldd 1024 --compute_type s --alpha 1.1 --beta 1 --batch_count 5
# BSS strided batched
$ ./rocblas-bench -f gemm_strided_batched_ex --transposeA N --transposeB T -m 1024 -n 2048 -k 512 --a_type bf16_r --lda 1024 --stride_a 4096 --b_type bf16_r --ldb 2048 --stride_b 4096 --c_type s --ldc 1024 --stride_c 2097152 --d_type s --ldd 1024 --stride_d 2097152 --compute_type s --alpha 1.1 --beta 1 --batch_count 5

# I8II
$ ./rocblas-bench -f gemm_ex --transposeA N --transposeB T -m 1024 -n 2048 -k 512 --a_type i8_r --lda 1024 --b_type i8_r --ldb 2048 --c_type i32_r --ldc 1024 --d_type i32_r --ldd 1024 --compute_type i32_r --alpha 1.1 --beta 1
# I8II batched
$ ./rocblas-bench -f gemm_batched_ex --transposeA N --transposeB T -m 1024 -n 2048 -k 512 --a_type i8_r --lda 1024 --b_type i8_r --ldb 2048 --c_type i32_r --ldc 1024 --d_type i32_r --ldd 1024 --compute_type i32_r --alpha 1.1 --beta 1 --batch_count 5
# I8II strided batched
$ ./rocblas-bench -f gemm_strided_batched_ex --transposeA N --transposeB T -m 1024 -n 2048 -k 512 --a_type i8_r --lda 1024 --stride_a 4096 --b_type i8_r --ldb 2048 --stride_b 4096 --c_type i32_r --ldc 1024 --stride_c 2097152 --d_type i32_r --ldd 1024 --stride_d 2097152 --compute_type i32_r --alpha 1.1 --beta 1 --batch_count 5
```

rocblas 调用过程中可以设置环境变量来配置文件日志记录：

- export ROCBLAS_LAYER=3　　# 启用 trace + bench + profile
- export ROCBLAS_LOG_PATH=./trace.log　# 指定日志文件输出路径

例如，测试 sgemm 性能的命令行如下：

```
./rocblas-bench -f gemm -r f32_r --transposeA N --transposeB N -m 4096 -n 4096 -k 4096 --alpha 1 --lda 4096 --ldb 4096 --beta 0 --ldc 4096
```

在 vega20 机器上上述命令，会产生一个结果信息如下：

```
transA,transB,M,N,K,alpha,lda,ldb,beta,ldc,rocblas-Gflops,us
N,N,4096,4096,4096,1,4096,4096,0,4096,11941.5,11509.4
```

其中性能结果为 11941.5Gflops。

6.2 rocblas-gemm-tune

rocblas-gemm-tune 用于为给定的一组 gemm size 找到性能最佳的 gemm kernel。通过程序生成的结果，用户可以通过配置文件的方式，更改程序中这些 GEMM 问题集所执行的 kernel。

```
执行命令： ./rocblas-gemm-tune --yaml test.yaml
```

其中 test.yaml 内容示例如下（ 可用 ROCBLAS_LAYER=4 抓取 ）：

```
- { rocblas_function： "rocblas_sgemm", transA： "N", transB： "N", M： 1024,
N： 2048, K： 512, lda： 1024, ldb： 2048, ldc： 1024, ldd： 1024}
```

预期输出：

```
transA,transB,M,N,batch_count,K,alpha,beta,lda,ldb,ldc,input_type,output_type,comp
ute_type,solution_index
N,N,1024,2048,1,512,1,0,1024,2048,1024,f32_r,f32_r,f32_r,14895
```

最后一列为 solution_index 是 rocblas 库中 GEMM 性能最好的 kernel 的索引，通过按照上述格式指定索引，即可改变该 SIZE 所执行的 kernel 。

如果输出存储在文件中，则结果可用于通过设置环境变量 export ROCBLAS_TENSILE_GEMM_OVERRIDE_PATH=<path>，其中<path>指向存储的文件，使用找到的内核覆盖默认内核选择。

6.3 rocblas-test

rocblas-test 用于执行 rocBLAS 单元测试，它使用 Googletest 框架。

测试分为五类：

1) quick
2) pre_checkin
3) nightly
4) stress
5) known_bug

执行 quick 测试的命令如下：

```
./rocblas-test --gtest_filter=*quick*
```

通过将 quick 替换为 pre_checkin、nightly 或者 known_bug 可以执行其他测试用例。

--gtest_filter 的模式是：

```
--gtest_filter=POSTIVE_PATTERNS[-NEGATIVE_PATTERNS]
```

gtest_filter 也可以用于为特定功能和特定输入参数集运行测试。 例如，运行 rocblas_saxpy 函数的所有快速测试：

```
./rocblas-test --gtest_filter=*quick*axpy*f32_r*
```

输出行数可以通过以下方式减少：

```
GTEST_LISTENER=NO_PASS_LINE_IN_LOG ./rocblas-test --gtest_filter=*quick*
```

7 Demo 案例

7.1 示例代码

demo_sgemm.cpp：

```
#include <iostream>
#include <hip/hip_runtime.h>
#include <rocblas/rocblas.h>

int main() {
    const int M = 2;
    const int N = 3;
    const int K = 4;

    float alpha = 1.0f;
    float beta  = 0.0f;

    float h_A[M*K] = {1, 2, 3, 4,
                      5, 6, 7, 8};  // A: M x K
    float h_B[K*N] = {1, 2, 3,
```

```
                        4, 5, 6,
                        7, 8, 9,
                        10,11,12};      // B: K x N
float h_C[M*N] = {0};                   // C: M x N (result)

float *d_A, *d_B, *d_C;

hipMalloc(&d_A, M*K * sizeof(float));
hipMalloc(&d_B, K*N * sizeof(float));
hipMalloc(&d_C, M*N * sizeof(float));

hipMemcpy(d_A, h_A, sizeof(h_A), hipMemcpyHostToDevice);
hipMemcpy(d_B, h_B, sizeof(h_B), hipMemcpyHostToDevice);
hipMemcpy(d_C, h_C, sizeof(h_C), hipMemcpyHostToDevice);

rocblas_handle handle;
rocblas_create_handle(&handle);

// Note: rocblas is column-major by default
rocblas_sgemm(handle,
                    rocblas_operation_none, rocblas_operation_none,
                    M, N, K,
                    &alpha,
                    d_A, M,
                    d_B, K,
                    &beta,
                    d_C, M);

hipMemcpy(h_C, d_C, sizeof(h_C), hipMemcpyDeviceToHost);
```

```
    std::cout << "Result matrix C:" << std::endl;
    for(int i = 0; i < M; ++i) {
        for(int j = 0; j < N; ++j) {
            std::cout << h_C[i + j*M] << " ";   // 列主序：i + j*M
        }
        std::cout << std::endl;
    }

    rocblas_destroy_handle(handle);
    hipFree(d_A);
    hipFree(d_B);
    hipFree(d_C);

    return 0;
}
```

7.2 编译指令

```
hipcc demo_sgemm.cpp -o demo_sgemm -lrocblas
```

7.3 输出结果

```
Result matrix C:
70 80 90
158 184 210
```

说明矩阵乘法执行成功。

保密声明

在此声明，该文档展示的全部技术信息及其相关内容，版权皆属于开发者社区（https://developer.sourcefind.cn/）所有。未经允许，严禁截屏、大规模传播及转发。另外，对使用该技术文档而导致任何侵犯第三方专利或其他权利的行为，开发者社区不承担任何责任。

感谢您的理解与支持。

开发者社区